

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ**



BÁO CÁO THỰC TẬP AI

BÀI 4: CNNs FOR IMAGE CLASSIFICATION

MÔN HỌC: THỰC HÀNH HỌC MÁY VÀ TRÍ TUỆ NHÂN TẠO

GIẢNG VIÊN HƯỚNG DẪN: PGS_TS. TRƯƠNG NGỌC SƠN

SINH VIÊN THỰC HIỆN: TRƯƠNG PHÚC THỊNH - 23161336

Tp. Hồ Chí Minh, tháng 5 năm 2026

MỤC LỤC

1. GIỚI THIỆU	1
1.1 Tổng quan về Convolutional Neural Networks	1
1.2 Giới thiệu bài toán	1
1.3 Dataset CIFAR-10	2
1.4 Mục tiêu bài thực hành	3
2. CƠ SỞ LÝ THUYẾT	4
2.1 Kiến trúc CNN	4
2.2 Convolution Layer	5
2.3 Activation Function ReLU.....	7
2.4 Feature Map	8
2.5 Pooling Layer và Max Pooling	9
2.6 Flatten Layer	10
2.7 Fully Connected Layer.....	12
2.8 Cross Entropy Loss.....	13
2.9 Adam Optimizer	15
Bước 1: Tính moment bậc nhất.....	16
Bước 2: Tính moment bậc hai.....	16
Bước 3: Hiệu chỉnh bias.....	17
Bước 4: Cập nhật trọng số.....	17
2.10 Dropout và Regularization.....	18
Khi Dropout quá lớn	20
2.11 Overfitting và Underfitting	21
2.12 Batch Size và Learning Rate.....	22
2.13 Learning Rate.....	25
2.14 Data Augmentation	27

2.15 Confusion Matrix	28
3. QUY TRÌNH THỰC HIỆN.....	29
3.1 Chuẩn bị dữ liệu.....	29
3.2 Xây dựng mô hình	29
3.3 Huấn luyện và đánh giá	29
3.4 Các tham số sử dụng.....	29
3.5 Công cụ và phần mềm sử dụng.....	30
4. KẾT QUẢ THỰC NGHIỆM.....	31
4.1 Chuẩn bị môi trường và dataset	31
4.1.1 Import thư viện và kiểm tra thiết bị	31
4.1.2 Tải dataset CIFAR-10	31
4.2 Bài tập 1: Dataset Exploration	33
4.2.1 Kiểm tra kích thước bộ dữ liệu	33
4.2.2 Hiển thị một số ảnh mẫu trong CIFAR-10.....	34
4.3 Bài tập 2: Phân tích kiến trúc CNN	34
4.3.1 Kiến trúc mô hình CNN cơ bản	34
4.3.2 Phân tích kích thước Feature Map và số lượng tham số	35
4.4 Bài tập 3: Thay đổi số Filter	37
4.4.1 Mục tiêu thí nghiệm	37
4.4.2 Kết quả thay đổi số Filter	38
4.5 Bài tập 4: Thêm Convolution Layer	40
4.5.1 Xây dựng mô hình CNN với 3 lớp Convolution.....	40
4.5.2 Kết quả thực nghiệm và đánh giá.....	41
4.6 Bài tập 5: Dropout.....	43
4.6.1 Thêm Dropout vào mô hình CNN.....	43

4.6.2 Kết quả thực nghiệm và đánh giá dropout	43
4.7 Bài tập 6: Batch Size.....	45
4.7.1 Code thử nghiệm các giá trị batch size	45
4.7.2 So sánh kết quả các giá trị Batch Size	45
4.8 Bài tập 7: Learning Rate	47
4.8.1 Code thử nghiệm các giá trị learning rate	47
4.8.2 Phân tích ảnh hưởng của Learning Rate	48
4.9 Bài tập 8: Data Augmentation	50
4.9.1 Thêm augmentation vào pipeline dữ liệu.....	50
4.9.2 Đánh giá hiệu quả của Data Augmentation	50
4.10 Bài tập 9: Confusion Matrix	52
4.10.1 Xây dựng và trực quan hóa Confusion Matrix	52
4.10.2 Phân tích kết quả Confusion Matrix	53
4.11 Bài tập 10: Cải thiện mô hình (Accuracy > 75%)	54
4.11.1 Chiến lược cải thiện	54
4.11.2 Kiến trúc BetterCNN	55
4.11.3 Kết quả đạt được	58
5. THẢO LUẬN VÀ ĐÁNH GIÁ	61
5.1 Ảnh hưởng của số Filter đến hiệu suất	61
5.2 Ảnh hưởng của số lớp Convolution.....	61
5.3 Hiệu quả của Dropout	61
5.4 Ảnh hưởng của Batch Size	62
5.5 Ảnh hưởng của Learning Rate	62
5.6 Hiệu quả của Data Augmentation.....	62
5.7 Phân tích Confusion Matrix.....	62

5.8 Tổng hợp cấu hình tốt nhất	63
5.9 Hạn chế và hướng cải thiện	64
6. KẾT LUẬN	66
6.1 Tóm tắt kết quả đạt được	66
6.2 Bài học rút ra.....	66
6.3 Hướng phát triển tiếp theo	67

MỤC LỤC HÌNH ẢNH

Hình 1: Tổng quan về Convolutional Neural Networks.....	1
Hình 2: Tổng quan về bài toán bộ dữ liệu CIFAR-10.....	2
Hình 3: Minh họa kiến trúc CNN	5
Hình 4: Minh họa mô tả Convolution Layer	6
Hình 5: Minh họa mô tả các hàm Activation Function	8
Hình 6: Minh họa mô tả future map trong convolution layer	9
Hình 7: Minh họa mô tả khác biệt max pooling và avg pooling	10
Hình 8: Mô tả quá trình Flatten layer	11
Hình 10: Minh họa cơ chế Dropout với các neuron được vô hiệu hóa ngẫu nhiên trong quá trình huấn luyện.....	21
Hình 13: Khởi tạo môi trường và kiểm tra GPU/CPU	31
Hình 14: Dataset CIFAR-10 được tải thành công	32
Hình 15: Kết quả in kích thước dataset (train: 50000, test: 10000)	33
Hình 16: 10 ảnh đầu tiên trong CIFAR-10.....	34
Hình 17. Kiến trúc CNN cơ bản sử dụng trong bài thực hành phân loại ảnh CIFAR-10	35
Hình 18: Kết quả huấn luyện với cấu hình Filter 16 → 32	38
Hình 19: Kết quả huấn luyện với cấu hình Filter 32 → 64	39
Hình 20: Kết quả huấn luyện với cấu hình Filter 64 → 128	39
Hình 21: Kết quả huấn luyện CNN thêm Conv3.....	42
Hình 22: Kết quả huấn luyện CNN với Dropout(p=0.5).....	44
Hình 23: Kết quả huấn luyện với Batch Size = 32	46
Hình 24: Kết quả huấn luyện với Batch Size = 64	46
Hình 25: Kết quả huấn luyện với Batch Size = 128	47
Hình 26: Biểu đồ Loss theo Learning Rate qua 10 epoch.....	49

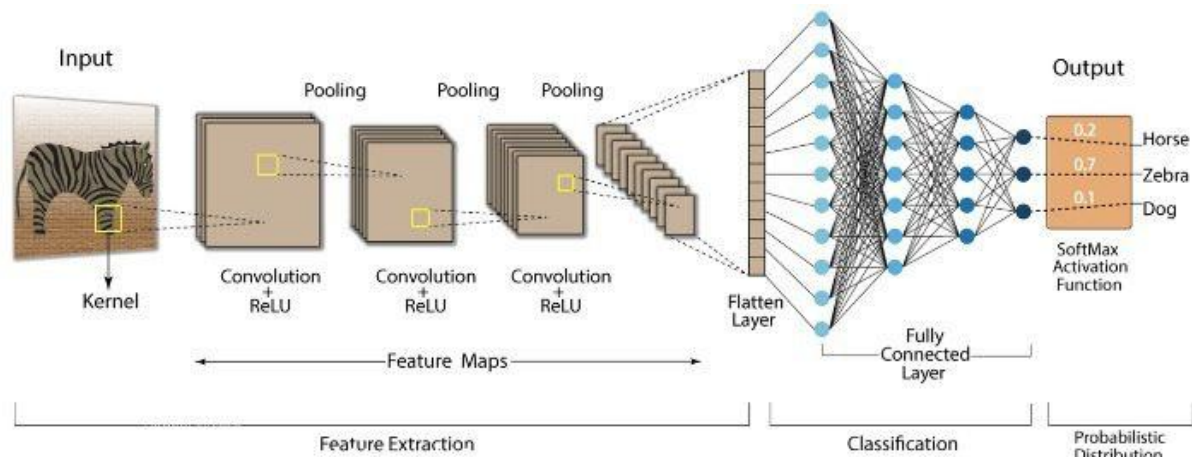
Hình 27: Kết quả huấn luyện CNN với Data Augmentation.....	51
Hình 28: Heatmap Confusion Matrix – CNN Base trên CIFAR-10	53
Hình 29: Loss Curve – BetterCNN qua 20 epoch.....	56
Hình 30: Loss Curve – BetterCNN	57
Hình 31: Confusion Matrix – BetterCNN	58
Hình 32: Bảng tổng hợp kết quả thực nghiệm.....	59

1. GIỚI THIỆU

1.1 Tổng quan về Convolutional Neural Networks

Convolutional Neural Networks (CNN) là một kiến trúc mạng neural network chuyên biệt được thiết kế để xử lý dữ liệu dạng lưới như ảnh và video. Khác với Multilayer Perceptron (MLP), CNN khai thác cấu trúc không gian của ảnh thông qua các lớp tích chập (convolution layers), giúp trích xuất đặc trưng cục bộ và bất biến theo vị trí.

CNN đã đạt được những bước tiến vượt bậc trong lĩnh vực Computer Vision, đặc biệt từ khi mô hình AlexNet giành chiến thắng tại cuộc thi ImageNet năm 2012. Ngày nay, CNN được ứng dụng rộng rãi trong nhận dạng ảnh, phát hiện đối tượng, phân đoạn ảnh và nhiều bài toán thị giác máy tính khác.



Hình 1: Tổng quan về Convolutional Neural Networks

1.2 Giới thiệu bài toán

Trong bài thực hành này, mô hình Convolutional Neural Network được sử dụng để giải quyết bài toán phân loại ảnh màu (Image Classification) trên bộ dữ liệu CIFAR-10. Đây là một trong những bộ dữ liệu chuẩn và phổ biến trong nghiên cứu học sâu, thường được sử dụng để đánh giá hiệu quả của các mô hình phân loại ảnh.



CIFAR-10 - Object Recognition in Images

Identify the subject of 60,000 labeled images

231 teams · 4 years ago

[Overview](#)
[Data](#)
[Discussion](#)
[Leaderboard](#)
[Rules](#)

Overview

Description	CIFAR-10 is an established computer-vision dataset used for object recognition. It is a subset of the 80 million tiny images dataset and consists of 60,000 32x32 color images containing one of 10 object classes, with 6000 images per class. It was collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.
Evaluation	

Hình 2: Tổng quan về bài toán bộ dữ liệu CIFAR-10

Bộ dữ liệu CIFAR-10 bao gồm 60.000 ảnh màu có kích thước 32×32 pixel, được chia thành 10 lớp đối tượng khác nhau gồm: airplane, automobile, bird, cat, deer, dog, frog, horse, ship và truck. Trong đó, 50.000 ảnh được sử dụng cho quá trình huấn luyện và 10.000 ảnh được sử dụng để kiểm tra mô hình.

Mục tiêu của bài thực hành là xây dựng và huấn luyện mô hình CNN bằng thư viện PyTorch để phân loại chính xác các ảnh trong bộ dữ liệu CIFAR-10. Bên cạnh đó, bài thực hành còn khảo sát ảnh hưởng của các yếu tố như số lượng bộ lọc (filters), số lớp tích chập, Dropout, Batch Size, Learning Rate và Data Augmentation đến hiệu suất của mô hình. Thông qua các thí nghiệm này, sinh viên có thể hiểu rõ hơn về nguyên lý hoạt động của CNN cũng như các kỹ thuật tối ưu hóa mô hình trong bài toán nhận dạng ảnh.

1.3 Dataset CIFAR-10

CIFAR-10 (Canadian Institute For Advanced Research) là một bộ dữ liệu chuẩn được sử dụng rộng rãi trong lĩnh vực Machine Learning và Deep Learning để nghiên cứu các bài toán phân loại ảnh. Bộ dữ liệu này được phát triển bởi nhóm nghiên cứu tại Đại học Toronto và thường được sử dụng để đánh giá hiệu suất của các mô hình nhận dạng ảnh.

Bộ dữ liệu CIFAR-10 có các đặc điểm chính như sau:

- Gồm 60.000 ảnh màu, trong đó có 50.000 ảnh huấn luyện và 10.000 ảnh kiểm tra.

- Kích thước mỗi ảnh là 32×32 pixel với 3 kênh màu RGB.
- Bao gồm 10 lớp đối tượng khác nhau: airplane, automobile, bird, cat, deer, dog, frog, horse, ship và truck.
- Dữ liệu được phân bố đồng đều với 6.000 ảnh cho mỗi lớp.

Các ảnh trong CIFAR-10 chứa nhiều đối tượng thực tế với sự thay đổi về góc nhìn, màu sắc, kích thước và bối cảnh nền, tạo nên độ phức tạp cao hơn so với nhiều bộ dữ liệu cơ bản khác.

So với bộ dữ liệu MNIST gồm các ảnh chữ số viết tay có kích thước 28×28 pixel và chỉ sử dụng một kênh màu xám (grayscale), CIFAR-10 khó xử lý hơn đáng kể do ảnh có ba kênh màu RGB và chứa các đối tượng thực tế đa dạng. Vì vậy, mô hình không chỉ cần học các đặc trưng hình dạng mà còn phải khai thác hiệu quả các đặc trưng về màu sắc, kết cấu và mối quan hệ không gian giữa các đối tượng trong ảnh. Điều này khiến CIFAR-10 trở thành một bộ dữ liệu phù hợp để đánh giá khả năng trích xuất đặc trưng của mạng Convolutional Neural Network (CNN).

1.4 Mục tiêu bài thực hành

Bài thực hành này được thực hiện nhằm giúp sinh viên hiểu và vận dụng kiến thức về mạng nơ-ron tích chập (Convolutional Neural Networks – CNN) trong bài toán phân loại ảnh. Các mục tiêu cụ thể bao gồm:

- Tìm hiểu kiến trúc và nguyên lý hoạt động của mạng Convolutional Neural Networks.
- Xây dựng và huấn luyện mô hình CNN bằng thư viện PyTorch cho bài toán phân loại ảnh trên bộ dữ liệu CIFAR-10.
- Khảo sát ảnh hưởng của các siêu tham số và kỹ thuật cải thiện mô hình như số lượng bộ lọc (filters), số lớp tích chập (convolution layers), Dropout, Batch Size, Learning Rate và Data Augmentation.
- Đánh giá hiệu suất mô hình thông qua độ chính xác (Accuracy) và ma trận nhầm lẫn (Confusion Matrix).

- So sánh kết quả giữa các mô hình và phân tích ảnh hưởng của từng yếu tố đến hiệu năng phân loại.
- Tối ưu hóa kiến trúc CNN nhằm đạt độ chính xác kiểm tra (Test Accuracy) lớn hơn 75% trên bộ dữ liệu CIFAR-10.
- Rèn luyện kỹ năng triển khai, huấn luyện và đánh giá mô hình học sâu trong môi trường thực tế bằng PyTorch.

2. CƠ SỞ LÝ THUYẾT

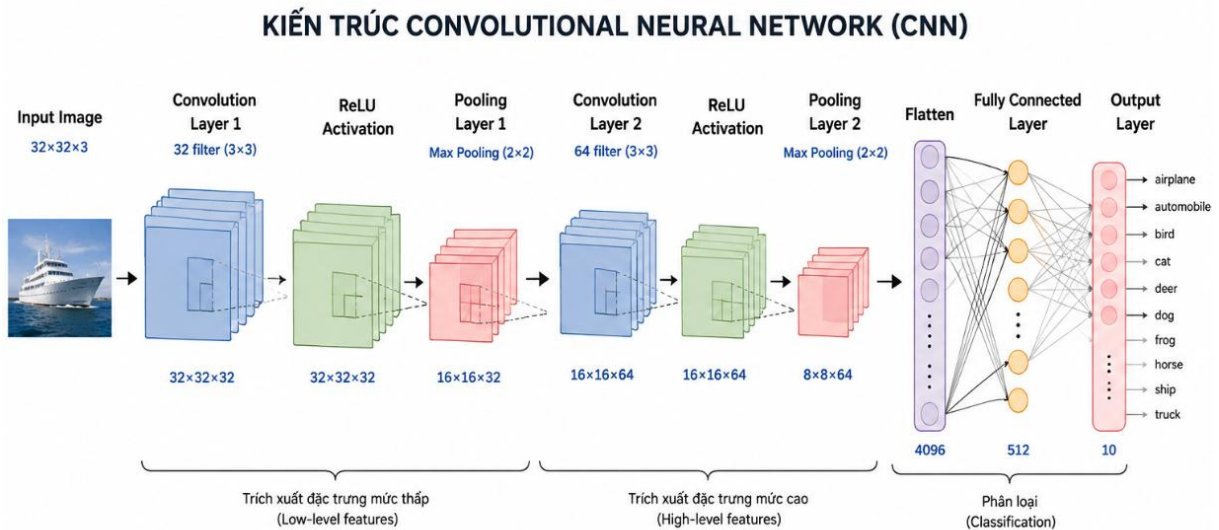
2.1 Kiến trúc CNN

Convolutional Neural Network (CNN) là một kiến trúc mạng nơ-ron sâu được thiết kế chuyên biệt cho dữ liệu có cấu trúc dạng lưới, đặc biệt là ảnh số. Khác với mạng nơ-ron truyền thống như Multilayer Perceptron (MLP), CNN không cần biến đổi toàn bộ ảnh thành một vector ngay từ đầu mà vẫn giữ được cấu trúc không gian hai chiều của ảnh. Điều này giúp mô hình khai thác tốt mối quan hệ giữa các điểm ảnh lân cận, từ đó học được các đặc trưng quan trọng phục vụ cho bài toán nhận dạng và phân loại ảnh.

Một kiến trúc CNN cơ bản thường bao gồm các thành phần chính: Convolution Layer, Activation Function, Pooling Layer, Flatten Layer và Fully Connected Layer. Trong đó, các lớp Convolution và Pooling đóng vai trò trích xuất đặc trưng, còn các lớp Fully Connected đảm nhiệm vai trò phân loại dựa trên các đặc trưng đã học được.

Trong bài thực hành này, mô hình CNN được xây dựng để phân loại ảnh trên bộ dữ liệu CIFAR-10. Ảnh đầu vào có kích thước 32×32 pixels với 3 kênh màu RGB. Sau khi đi qua các lớp tích chập và pooling, ảnh được chuyển thành vector đặc trưng rồi đưa vào các lớp phân loại để dự đoán một trong 10 lớp đối tượng.

Kiến trúc CNN tổng quát có thể mô tả như sau:



Hình 3: Minh họa kiến trúc CNN

Ưu điểm của CNN trong xử lý ảnh gồm:

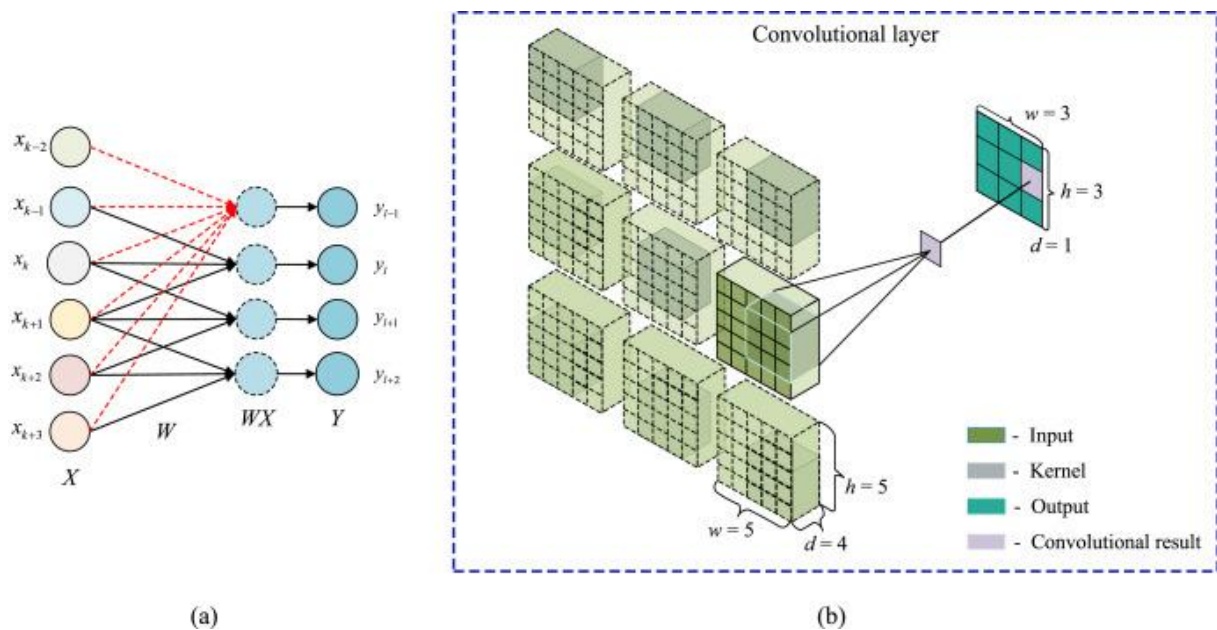
- Khai thác tốt cấu trúc không gian của ảnh.
- Giảm số lượng tham số nhờ cơ chế chia sẻ trọng số.
- Có khả năng học đặc trưng tự động thay vì phải thiết kế đặc trưng thủ công.
- Phù hợp với các bài toán phân loại ảnh, nhận dạng đối tượng và thị giác máy tính.

2.2 Convolution Layer

Convolution Layer là thành phần cốt lõi và quan trọng nhất của CNN. Lớp này sử dụng các bộ lọc, còn gọi là filter hoặc kernel, để quét qua ảnh đầu vào và thực hiện phép tích chập. Mục đích của quá trình này là trích xuất các đặc trưng cục bộ trong ảnh như cạnh, góc, đường cong, màu sắc, kết cấu bề mặt hoặc các chi tiết hình học.

Mỗi filter có kích thước nhỏ, thường là 3×3 hoặc 5×5 . Filter sẽ lần lượt di chuyển qua ảnh theo từng bước xác định và tính toán tích vô hướng giữa các giá trị pixel trong vùng ảnh với trọng số của filter. Kết quả thu được là một Feature Map, biểu diễn mức độ xuất hiện của một đặc trưng nào đó tại các vị trí khác nhau trong ảnh.

Trong CNN, mỗi filter học một loại đặc trưng khác nhau. Ví dụ:



Hình 4: Minh họa mô tả Convolution Layer

- Filter thứ nhất có thể học đặc trưng cạnh ngang.
- Filter thứ hai có thể học đặc trưng cạnh dọc.
- Filter thứ ba có thể học đặc trưng góc hoặc đường cong.

Ở các lớp sâu hơn, filter có thể học các đặc trưng phức tạp hơn như mắt, bánh xe, đầu động vật hoặc hình dạng tổng thể của đối tượng.

Các tham số quan trọng của Convolution Layer gồm:

- **in_channels**: số kênh đầu vào. Với ảnh màu RGB, giá trị này bằng 3.
- **out_channels**: số lượng filter được sử dụng, đồng thời cũng là số lượng feature map đầu ra.
- **kernel_size**: kích thước của filter, ví dụ 3×3 .
- **stride**: bước dịch chuyển của filter trên ảnh.
- **padding**: số pixel được thêm vào biên ảnh để kiểm soát kích thước đầu ra.

Trong bài thực hành, lớp Convolution đầu tiên nhận ảnh đầu vào có 3 kênh màu và sử dụng 32 filter kích thước 3×3 . Do đó, lớp này tạo ra 32 feature map đầu ra. Lớp Convolution thứ hai nhận 32 feature map từ lớp trước và tạo ra 64 feature map mới.

Việc tăng số lượng filter ở các lớp sau giúp mô hình học được nhiều đặc trưng phức tạp hơn.

2.3 Activation Function ReLU

Sau mỗi lớp Convolution, hàm kích hoạt ReLU thường được sử dụng để đưa tính phi tuyến vào mô hình. Nếu chỉ sử dụng các phép biến đổi tuyến tính, dù mạng có nhiều lớp thì toàn bộ mô hình vẫn chỉ tương đương với một phép biến đổi tuyến tính duy nhất. Khi đó, mạng sẽ không đủ khả năng học các mối quan hệ phức tạp trong dữ liệu ảnh.

ReLU là viết tắt của Rectified Linear Unit. Hàm này giữ nguyên các giá trị dương và đưa các giá trị âm về 0. Có thể hiểu đơn giản rằng ReLU chỉ cho các tín hiệu tích cực đi qua và loại bỏ các tín hiệu âm.

Hàm ReLU có dạng:

$$f(x) = \max(0, x)$$

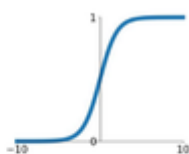
Đặc điểm của ReLU:

- Khi $(x < 0)$, đầu ra bằng 0
- Khi $(x > 0)$, đầu ra bằng chính giá trị đầu vào

Activation Functions

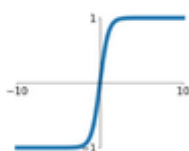
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



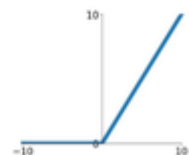
tanh

$$\tanh(x)$$



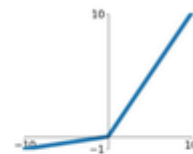
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

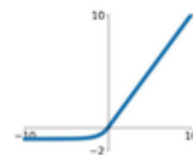


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Hình 5: Minh họa mô tả các hàm Activation Function

Điều này giúp ReLU giảm đáng kể hiện tượng vanishing gradient so với các hàm kích hoạt truyền thống như Sigmoid hoặc Tanh.

Ưu điểm của ReLU gồm:

- Công thức đơn giản, tốc độ tính toán nhanh.
- Giúp mô hình học nhanh hơn so với Sigmoid hoặc Tanh.
- Giảm hiện tượng Vanishing Gradient.
- Phù hợp với các mạng CNN có nhiều lớp.

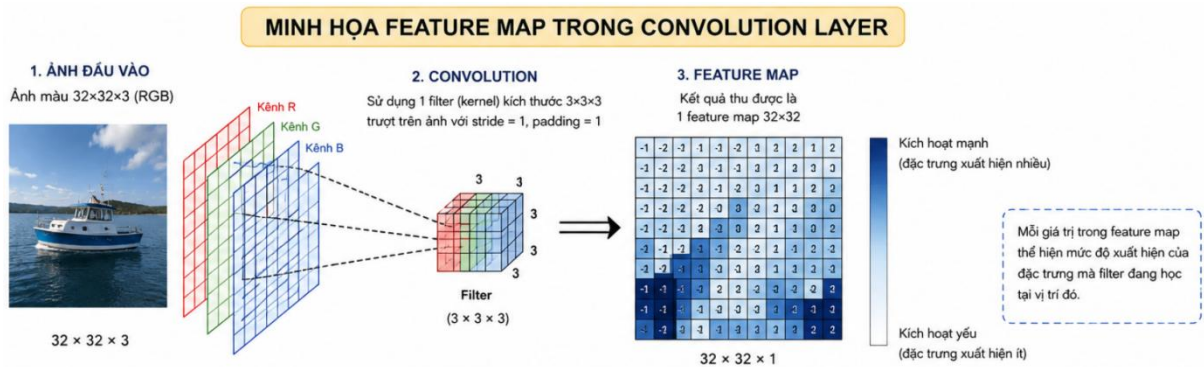
Trong bài thực hành này, ReLU được đặt sau mỗi lớp Convolution và sau lớp Fully Connected đầu tiên. Nhờ đó, mô hình có khả năng học các đặc trưng phi tuyến từ ảnh CIFAR-10, vốn là bộ dữ liệu có nhiều biến đổi về màu sắc, hình dạng và nền ảnh.

2.4 Feature Map

Feature Map (bản đồ đặc trưng) là đầu ra thu được sau khi dữ liệu đi qua một bộ lọc (filter) trong lớp Convolution. Mỗi Feature Map thể hiện mức độ xuất hiện của một đặc trưng cụ thể tại các vị trí khác nhau trong ảnh. Nói cách khác, Feature Map cho biết đặc trưng mà bộ lọc đang học xuất hiện mạnh hay yếu ở từng vùng của ảnh đầu vào.

Trong mạng CNN, mỗi filter sẽ tạo ra một Feature Map riêng. Do đó, nếu một lớp Convolution sử dụng 32 filter thì đầu ra của lớp đó sẽ gồm 32 Feature Map. Mỗi Feature Map tương ứng với một loại đặc trưng khác nhau được mô hình học trong quá trình huấn luyện.

Ở các lớp Convolution đầu tiên, Feature Map thường biểu diễn các đặc trưng mức thấp như cạnh, góc, đường thẳng, màu sắc hoặc kết cấu bề mặt. Khi đi sâu hơn vào mạng, các Feature Map sẽ biểu diễn các đặc trưng mức cao hơn như bộ phận của đối tượng hoặc hình dạng tổng thể của vật thể. Quá trình này giúp CNN xây dựng biểu diễn dữ liệu theo từng cấp độ từ đơn giản đến phức tạp.



Hình 6: Minh họa mô tả feature map trong convolution layer

Trong bài toán phân loại ảnh CIFAR-10, Feature Map đóng vai trò rất quan trọng vì toàn bộ thông tin đầu vào đều được chuyển đổi và biểu diễn dưới dạng các đặc trưng này trước khi đưa vào các lớp phân loại. Nếu các Feature Map không thể hiện được đầy đủ đặc điểm của ảnh, các lớp Fully Connected phía sau sẽ gặp khó khăn trong việc phân biệt các lớp đối tượng khác nhau. Vì vậy, số lượng filter, số lớp Convolution và kiến trúc mạng CNN có ảnh hưởng trực tiếp đến chất lượng Feature Map cũng như độ chính xác cuối cùng của mô hình phân loại.

2.5 Pooling Layer và Max Pooling

Pooling Layer được sử dụng sau lớp Convolution và ReLU nhằm giảm kích thước không gian của feature map. Việc giảm kích thước này giúp giảm số lượng phép tính, giảm số lượng tham số và hạn chế hiện tượng overfitting.

Phương pháp pooling phổ biến nhất là Max Pooling. Max Pooling lấy giá trị lớn nhất trong một vùng nhỏ của feature map. Ví dụ, với vùng 2×2 , chỉ giá trị lớn nhất được giữ lại và các giá trị còn lại bị loại bỏ.

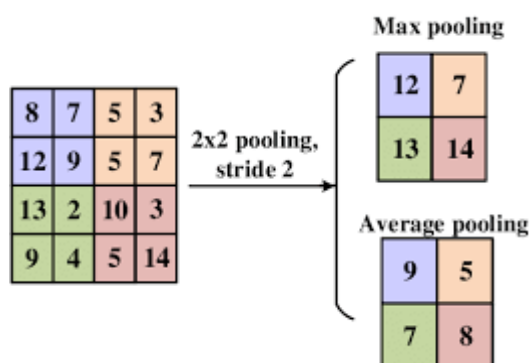
Ví dụ:

1 3

2 5

Sau Max Pooling, kết quả là: 5

Ý nghĩa của Max Pooling là giữ lại đặc trưng nổi bật nhất trong vùng đang xét. Nếu một filter phát hiện cạnh hoặc chi tiết quan trọng tại một vùng nào đó, Max Pooling sẽ giữ lại giá trị mạnh nhất đại diện cho đặc trưng đó.



Hình 7: Minh họa mô tả khác biệt max pooling và avg pooling

Trong bài thực hành, MaxPool2d với `kernel_size = 2` và `stride = 2` được sử dụng. Điều này làm kích thước feature map giảm một nửa sau mỗi lần pooling:

$$32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$$

Việc giảm kích thước feature map giúp mô hình tính toán nhanh hơn nhưng vẫn giữ lại các thông tin quan trọng phục vụ cho phân loại.

2.6 Flatten Layer

Sau khi đi qua các lớp Convolution và Pooling, dữ liệu đầu ra vẫn có dạng tensor nhiều chiều, bao gồm chiều cao (Height), chiều rộng (Width) và số kênh (Channels). Tuy nhiên, các lớp Fully Connected chỉ có thể xử lý dữ liệu ở dạng vector một chiều. Vì vậy, cần sử dụng Flatten Layer để chuyển đổi tensor nhiều chiều thành vector một chiều trước khi đưa vào giai đoạn phân loại.

Ví dụ, trong mô hình CNN được sử dụng trong bài thực hành, sau hai lớp Convolution và hai lớp Max Pooling, Feature Map cuối cùng có kích thước:

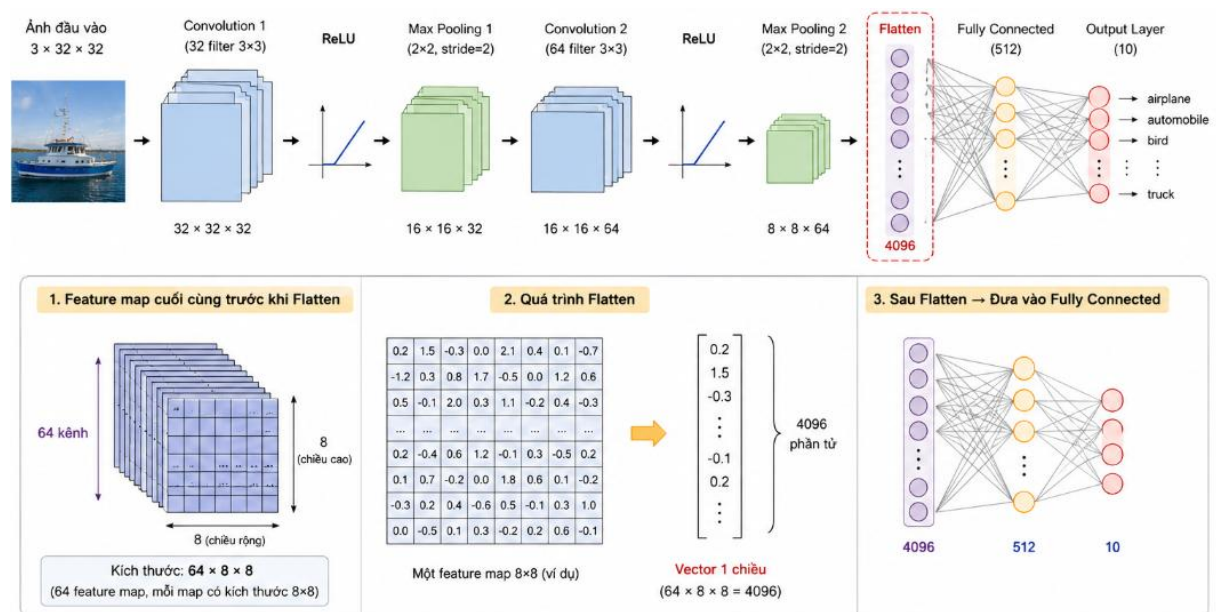
$$[64 \times 8 \times 8]$$

Điều này có nghĩa là mô hình đã trích xuất được 64 Feature Map, mỗi Feature Map có kích thước 8×8 . Khi đi qua Flatten Layer, toàn bộ dữ liệu này được trải thành một vector một chiều với số phần tử:

$$[64 \times 8 \times 8 = 4096]$$

Vector 4096 chiều này chứa toàn bộ thông tin đặc trưng đã được các lớp Convolution và Pooling trích xuất từ ảnh đầu vào. Sau đó, vector được đưa vào các lớp Fully Connected để thực hiện quá trình phân loại.

Cần lưu ý rằng Flatten Layer không thực hiện bất kỳ phép tính học nào và không làm thay đổi giá trị dữ liệu. Lớp này chỉ thay đổi hình dạng (shape) của dữ liệu từ tensor nhiều chiều sang vector một chiều. Mặc dù đơn giản, Flatten Layer đóng vai trò là cầu nối quan trọng giữa phần trích xuất đặc trưng của CNN và phần phân loại ở cuối mạng.



Hình 8: Mô tả quá trình Flatten layer

Trong PyTorch, Flatten thường được thực hiện bằng lệnh:

```
x = x.view(-1, 64 * 8 * 8)
```

Lệnh này chuyển toàn bộ Feature Map có kích thước $64 \times 8 \times 8$ thành vector 4096 phần tử để đưa vào lớp Fully Connected đầu tiên.

2.7 Fully Connected Layer

Fully Connected Layer (FC Layer) là lớp kết nối đầy đủ, trong đó mỗi neuron của lớp trước được kết nối với tất cả các neuron của lớp sau. Đây là phần cuối của mạng CNN, có nhiệm vụ sử dụng các đặc trưng đã được trích xuất từ các lớp Convolution và Pooling để thực hiện quá trình phân loại.

Sau khi đi qua Flatten Layer, toàn bộ thông tin đặc trưng của ảnh được biểu diễn dưới dạng một vector một chiều. Vector này được đưa vào các lớp Fully Connected để học mối quan hệ giữa các đặc trưng và các lớp đầu ra. Có thể xem Fully Connected Layer như một bộ phân loại (Classifier) hoạt động dựa trên các đặc trưng đã được CNN trích xuất.

Trong bài thực hành, mô hình CNN gốc sử dụng hai lớp Fully Connected:

$$\text{FC1: } 4096 \rightarrow 512$$

$$\text{FC2: } 512 \rightarrow 10$$

Lớp FC1 nhận vector đặc trưng có kích thước 4096 và chuyển đổi thành 512 đặc trưng cấp cao hơn. Quá trình này giúp mô hình học được các mối quan hệ phức tạp giữa các đặc trưng đã được trích xuất từ ảnh.

Lớp FC2 nhận đầu vào gồm 512 đặc trưng và tạo ra 10 giá trị đầu ra tương ứng với 10 lớp của bộ dữ liệu CIFAR-10. Các giá trị này thường được gọi là logits và biểu thị mức độ tin cậy của mô hình đối với từng lớp.

Trong quá trình dự đoán, lớp có giá trị đầu ra lớn nhất sẽ được chọn làm kết quả phân loại cuối cùng. Ví dụ, nếu đầu ra của mô hình là:

$$[0.2, ; 1.5, ; 0.8, ; 4.3, ; 0.1, \dots]$$

thì lớp có giá trị 4.3 sẽ được xem là lớp được dự đoán với độ tin cậy cao nhất.

Mặc dù đóng vai trò quan trọng trong việc phân loại, Fully Connected Layer thường chứa số lượng tham số rất lớn do mọi neuron đều được kết nối với nhau. Trong mô hình CNN của bài thực hành, phần lớn tham số của mạng tập trung ở lớp FC1. Vì vậy, nếu số lượng neuron quá lớn, mô hình có thể gặp hiện tượng overfitting, làm giảm khả năng tổng quát hóa trên dữ liệu mới. Đây cũng là lý do các kiến trúc CNN hiện đại thường sử dụng Dropout hoặc các kỹ thuật giảm số lượng tham số để cải thiện hiệu quả huấn luyện.

2.8 Cross Entropy Loss

Trong các bài toán phân loại đa lớp (multi-class classification), hàm mất mát đóng vai trò đánh giá mức độ sai lệch giữa kết quả dự đoán của mô hình và nhãn thực tế. Một mô hình có giá trị loss càng nhỏ thì khả năng dự đoán càng chính xác.

Trong bài thực hành này, hàm mất mát Cross Entropy Loss được sử dụng để huấn luyện mô hình CNN trên dataset CIFAR-10. Đây là hàm mất mát phổ biến và hiệu quả nhất cho các bài toán phân loại nhiều lớp trong Deep Learning.

Cross Entropy Loss đo mức độ khác biệt giữa phân phối xác suất dự đoán của mô hình và phân phối xác suất thực tế của dữ liệu.

Với:

- (N): số lượng mẫu dữ liệu
- (C): số lớp phân loại

Hàm Cross Entropy Loss được định nghĩa như sau:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c})$$

Trong đó:

- $(y_{i,c})$: nhãn thực tế của mẫu thứ (i) tại lớp (c)
- $(p_{i,c})$: xác suất dự đoán của mô hình cho lớp (c)

- (L): giá trị hàm mất mát

Trong bài toán phân loại, nhãn thực tế thường được biểu diễn dưới dạng one-hot vector.

Ví dụ với bài toán CIFAR-10 gồm 10 lớp. Nếu ảnh là chữ số 3 thì vector nhãn sẽ là:

[0,0,0,1,0,0,0,0,0,0]

Sau khi dữ liệu đi qua output layer, mô hình sử dụng hàm Softmax để chuyển đổi đầu ra thành xác suất dự đoán cho từng lớp:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Hàm Softmax đảm bảo:

- Tổng các xác suất bằng 1
- Lớp có xác suất lớn nhất sẽ là kết quả dự đoán cuối cùng

Cross Entropy Loss hoạt động bằng cách:

- tăng loss khi mô hình dự đoán sai
- giảm loss khi mô hình dự đoán đúng với xác suất cao

Ví dụ:

- Nếu mô hình dự đoán đúng lớp với xác suất 0.99 thì loss rất nhỏ
- Nếu mô hình dự đoán sai hoặc xác suất lớp đúng thấp thì loss sẽ lớn

Do đó, quá trình huấn luyện sẽ cố gắng cập nhật trọng số nhằm giảm giá trị loss qua từng epoch.

Cross Entropy Loss thường được kết hợp với Softmax activation ở output layer vì:

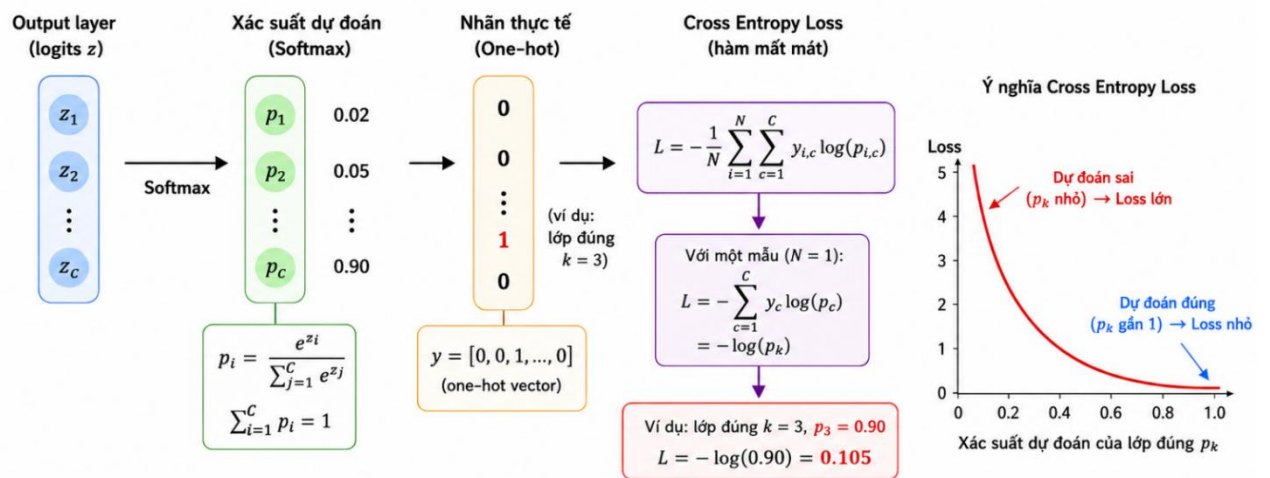
- Softmax chuyển đầu ra thành phân phối xác suất
- Cross Entropy đánh giá độ sai khác giữa xác suất dự đoán và nhãn thực tế

Sự kết hợp này giúp quá trình huấn luyện ổn định, gradient rõ ràng và mô hình hội tụ nhanh hơn.

Trong PyTorch, Cross Entropy Loss được sử dụng thông qua:

```
criterion = nn.CrossEntropyLoss()
```

Hàm này đã tích hợp cả Softmax và Cross Entropy trong cùng một phép tính, giúp tối ưu hiệu năng và tránh sai số số học trong quá trình huấn luyện.



Hình 9: Minh họa Softmax và Cross Entropy Loss trong mạng neural

2.9 Adam Optimizer

Trong quá trình huấn luyện mạng neural network, sau khi thuật toán Backpropagation tính được gradient của hàm mất mát, mô hình cần một phương pháp để cập nhật các trọng số nhằm giảm sai số dự đoán. Quá trình này được thực hiện bởi thuật toán tối ưu hóa (Optimizer).

Trong bài thực hành này, thuật toán Adam (Adaptive Moment Estimation) được sử dụng để tối ưu mô hình CNN. Adam là một trong những optimizer phổ biến nhất trong Deep Learning nhờ khả năng hội tụ nhanh, ổn định và hoạt động hiệu quả trên nhiều loại dữ liệu khác nhau.

Adam được phát triển dựa trên sự kết hợp ưu điểm của:

- AdaGrad: thích nghi learning rate cho từng tham số
- RMSProp: giảm dao động gradient và cải thiện tốc độ hội tụ

Khác với SGD (Stochastic Gradient Descent) chỉ sử dụng gradient hiện tại để cập nhật trọng số, Adam sử dụng:

- moment bậc nhất (First Moment): trung bình động của gradient
- moment bậc hai (Second Moment): trung bình động của bình phương gradient

Điều này giúp Adam:

- cập nhật trọng số ổn định hơn
- giảm dao động trong quá trình học
- hội tụ nhanh hơn SGD truyền thống

Quá trình cập nhật của Adam gồm các bước:

Bước 1: Tính moment bậc nhất

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Bước 2: Tính moment bậc hai

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Trong đó:

- (g_t) : gradient tại bước cập nhật thứ (t)
- (m_t) : moment bậc nhất
- (v_t) : moment bậc hai
- (β_1, β_2) : hệ số suy giảm

Do (m_t) và (v_t) ban đầu có xu hướng lệch về 0, Adam sử dụng thêm bước hiệu chỉnh bias:

Bước 3: Hiệu chỉnh bias

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Bước 4: Cập nhật trọng số

$$\theta_t = \theta_{t-1} - \frac{\alpha \widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

Trong đó:

- (θ_t) : trọng số tại bước (t)
- (α) : learning rate
- (ϵ) : hằng số nhỏ giúp tránh chia cho 0

Các siêu tham số mặc định của Adam thường được sử dụng:

- $(\beta_1 = 0.9)$
- $(\beta_2 = 0.999)$
- $(\epsilon = 10^{-8})$

Ưu điểm của Adam:

- Hội tụ nhanh
- Hoạt động ổn định trên dữ liệu lớn
- Tự động điều chỉnh learning rate cho từng tham số
- Hiệu quả với mạng neural sâu

Nhược điểm:

- Tốn bộ nhớ hơn SGD
- Đôi khi tổng quát hóa (generalization) kém hơn SGD trên một số bài toán lớn

Trong bài thực hành này, Adam giúp mô hình CNN huấn luyện ổn định và đạt accuracy cao trên tập dữ liệu CIFAR-10.

Trong PyTorch, Adam optimizer được sử dụng như sau:

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

2.10 Dropout và Regularization

Trong Deep Learning, một trong những vấn đề phổ biến nhất là hiện tượng overfitting. Overfitting xảy ra khi mô hình học quá chi tiết trên tập huấn luyện, bao gồm cả nhiễu và các đặc trưng không quan trọng. Khi đó, mô hình đạt accuracy rất cao trên tập train nhưng hoạt động kém trên tập test hoặc dữ liệu mới.

Để giảm hiện tượng này, nhiều kỹ thuật regularization được sử dụng nhằm cải thiện khả năng tổng quát hóa (generalization) của mô hình. Một trong những kỹ thuật regularization phổ biến và hiệu quả nhất là Dropout.

Dropout được đề xuất bởi Srivastava et al. (2014) và được sử dụng rộng rãi trong các mô hình Deep Learning hiện đại.

Nguyên lý hoạt động của Dropout

Trong quá trình huấn luyện, Dropout sẽ ngẫu nhiên “tắt” một số neuron trong mạng neural với xác suất (p). Khi một neuron bị tắt:

- giá trị đầu ra của neuron sẽ bằng 0
- neuron đó tạm thời không tham gia vào quá trình lan truyền thuận và lan truyền ngược

Ví dụ: Dropout = 0.5 nghĩa là khoảng 50% neuron sẽ bị vô hiệu hóa ngẫu nhiên trong mỗi lần huấn luyện.

Điều này giúp:

- giảm sự phụ thuộc quá mức giữa các neuron
- ngăn mô hình ghi nhớ dữ liệu huấn luyện
- tăng khả năng học đặc trưng tổng quát

Dropout có thể được biểu diễn như sau:

$$a_i^{drop} = m_i \cdot a_i$$

Trong đó:

- (a_i) : đầu ra của neuron
- (m_i) : biến Bernoulli ngẫu nhiên
- $(m_i = 0)$: neuron bị tắt
- $(m_i = 1)$: neuron hoạt động bình thường

Dropout trong giai đoạn huấn luyện

Trong mỗi iteration:

- một tập neuron khác nhau sẽ bị vô hiệu hóa ngẫu nhiên
- mạng neural thực tế sẽ thay đổi cấu trúc liên tục

Điều này khiến mô hình:

- không thể phụ thuộc vào một neuron cụ thể
- buộc phải học các đặc trưng ổn định hơn
- hoạt động tương tự như việc huấn luyện nhiều mạng neural nhỏ cùng lúc

Dropout giúp:

- giảm overfitting
- tăng khả năng tổng quát hóa
- cải thiện hiệu suất trên tập test

Dropout trong giai đoạn dự đoán (Inference)

Trong giai đoạn inference hoặc testing:

- tất cả neuron đều hoạt động
- không còn neuron nào bị tắt

Tuy nhiên, để đảm bảo giá trị đầu ra nhất quán với quá trình huấn luyện, đầu ra của neuron thường được nhân với: $(1-p)$ hoặc được framework tự động scale trong quá trình training.

Điều này giúp:

- giữ ổn định phân phối dữ liệu
- tránh thay đổi quá lớn giữa training và inference

Ảnh hưởng của Dropout khi Dropout quá nhỏ

- hiệu quả regularization thấp
- mô hình vẫn dễ bị overfitting

Khi Dropout quá lớn

- quá nhiều neuron bị vô hiệu hóa
- mô hình khó học đặc trưng
- accuracy có thể giảm

Trong thực tế:

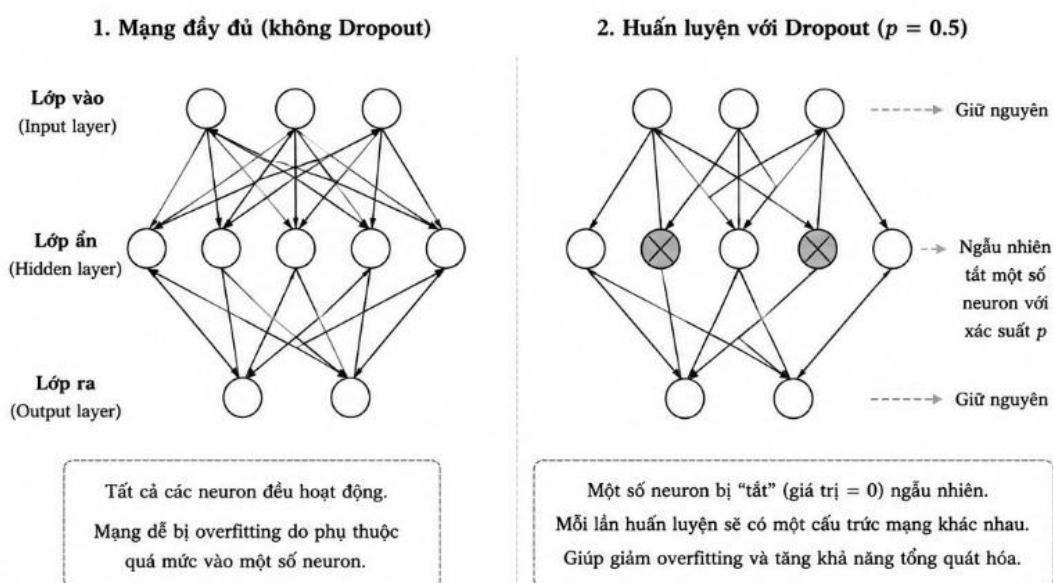
- Dropout = 0.2 đến 0.5 thường được sử dụng phổ biến

Trong bài thực hành này, Dropout giúp mô hình CNN:

- giảm overfitting
- cải thiện khả năng tổng quát hóa
- đạt accuracy ổn định hơn trên tập test CIFAR-10

Trong PyTorch, Dropout được sử dụng như sau:

```
nn.Dropout (p=0.5)
```



Hình 10: Minh họa cơ chế Dropout với các neuron được vô hiệu hóa ngẫu nhiên trong quá trình huấn luyện

2.11 Overfitting và Underfitting

Trong quá trình huấn luyện mô hình học máy và học sâu, hai hiện tượng thường gặp là **Overfitting** và **Underfitting**. Đây là những vấn đề ảnh hưởng trực tiếp đến khả năng tổng quát hóa của mô hình trên dữ liệu mới.

Overfitting xảy ra khi mô hình học quá chi tiết trên tập dữ liệu huấn luyện. Khi đó, mô hình không chỉ học các đặc trưng quan trọng mà còn ghi nhớ cả nhiễu và các đặc điểm riêng biệt của tập train. Kết quả là mô hình đạt hiệu suất rất cao trên dữ liệu huấn luyện nhưng lại hoạt động kém trên dữ liệu kiểm tra hoặc dữ liệu thực tế.

Dấu hiệu nhận biết Overfitting:

- Training Accuracy rất cao.
- Test Accuracy thấp hơn đáng kể so với Training Accuracy.
- Training Loss tiếp tục giảm nhưng Validation/Test Loss bắt đầu tăng.
- Khoảng cách giữa kết quả trên tập train và tập test ngày càng lớn.

Ngược lại, **Underfitting** xảy ra khi mô hình quá đơn giản hoặc chưa được huấn luyện đầy đủ, khiến mô hình không thể học được các đặc trưng quan trọng của dữ liệu. Trong trường hợp này, mô hình hoạt động kém trên cả tập huấn luyện và tập kiểm tra.

Dấu hiệu nhận biết Underfitting:

- Training Accuracy thấp.
- Test Accuracy thấp.
- Training Loss và Test Loss đều cao.
- Mô hình không thể khai thác được các đặc trưng cần thiết của dữ liệu.

Trong bài thực hành phân loại ảnh CIFAR-10 bằng CNN, hiện tượng Overfitting có thể xuất hiện khi tăng quá nhiều số lượng filter, thêm nhiều lớp Convolution hoặc huấn luyện với số epoch lớn. Để giảm Overfitting, có thể áp dụng các kỹ thuật như Dropout, Data Augmentation, Early Stopping hoặc giảm độ phức tạp của mô hình.

Ngược lại, Underfitting có thể xảy ra khi mô hình quá đơn giản, số lượng filter quá ít hoặc thời gian huấn luyện chưa đủ. Khi đó, cần tăng khả năng biểu diễn của mô hình bằng cách bổ sung lớp Convolution, tăng số lượng filter hoặc huấn luyện thêm nhiều epoch.

Việc cân bằng giữa Overfitting và Underfitting là một trong những mục tiêu quan trọng trong quá trình thiết kế và tối ưu hóa mô hình CNN nhằm đạt được khả năng tổng quát hóa tốt nhất trên dữ liệu kiểm tra.

2.12 Batch Size và Learning Rate

Trong quá trình huấn luyện mạng neural network, dữ liệu huấn luyện thường không được đưa toàn bộ vào mô hình cùng một lúc mà được chia thành nhiều nhóm nhỏ gọi là batch. Batch size là số lượng mẫu dữ liệu được xử lý trong một lần cập nhật trọng số của mô hình.

Ví dụ:

- Batch size = 32 nghĩa là mô hình sẽ xử lý 32 ảnh trước khi thực hiện cập nhật trọng số một lần.
- Nếu tập huấn luyện MNIST có 60.000 ảnh và batch size = 64, số lần cập nhật trọng số trong một epoch sẽ là:

$$\frac{60000}{64} \approx 937$$

Batch size ảnh hưởng trực tiếp đến:

- tốc độ huấn luyện
- độ ổn định của gradient
- khả năng hội tụ của mô hình
- mức sử dụng bộ nhớ GPU/CPU

Batch Size nhỏ

Khi batch size nhỏ (ví dụ: 32):

- Mô hình cập nhật trọng số thường xuyên hơn
- Gradient có độ dao động lớn hơn do chỉ dựa trên một lượng dữ liệu nhỏ
- Giúp mô hình dễ thoát khỏi local minima
- Khả năng tổng quát hóa (generalization) thường tốt hơn

Tuy nhiên:

- Tốc độ huấn luyện chậm hơn
- Thời gian training kéo dài
- GPU hoạt động chưa tối ưu

Batch size nhỏ thường phù hợp với:

- dữ liệu phức tạp
- mô hình lớn
- hệ thống có bộ nhớ hạn chế

Batch Size lớn

Khi batch size lớn (ví dụ: 128):

- Gradient ổn định hơn do được tính trên nhiều mẫu dữ liệu
- Tận dụng tốt khả năng xử lý song song của GPU
- Tốc độ huấn luyện nhanh hơn

Tuy nhiên:

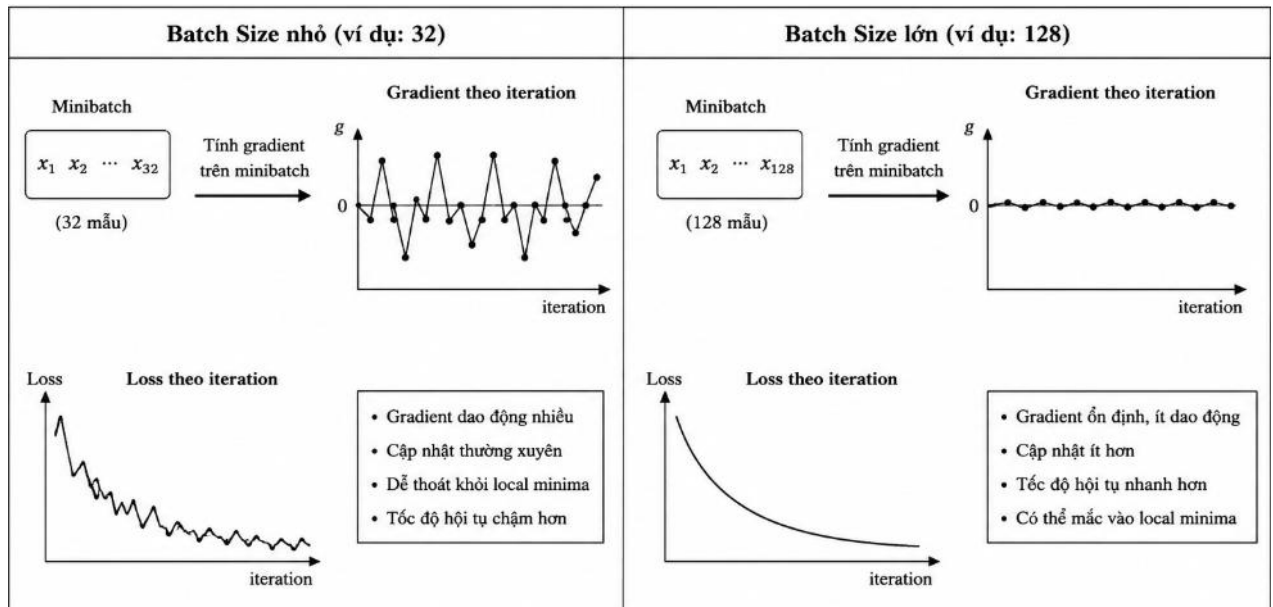
- Mô hình có thể bị mắc tại local minima hoặc saddle point
- Khả năng tổng quát hóa đôi khi giảm
- Yêu cầu nhiều bộ nhớ hơn

Batch size quá lớn cũng có thể làm mô hình hội tụ kém linh hoạt vì gradient thay đổi ít qua từng lần cập nhật.

Ngoài ra, batch size còn ảnh hưởng đến số lần cập nhật trọng số trong mỗi epoch:

- Batch size nhỏ → nhiều lần cập nhật hơn
- Batch size lớn → ít lần cập nhật hơn

Điều này ảnh hưởng trực tiếp đến quá trình hội tụ của mô hình trong huấn luyện Deep Learning.



Hình 11: So sánh Batch Size nhỏ và Batch Size lớn trong quá trình cập nhật gradient

2.13 Learning Rate

Learning Rate (ký hiệu: η hoặc α) là một siêu tham số quan trọng trong quá trình huấn luyện mạng neural network. Learning rate quyết định mức độ thay đổi của trọng số sau mỗi lần cập nhật gradient trong quá trình tối ưu hóa.

Trong thuật toán Gradient Descent, trọng số được cập nhật theo công thức:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Trong đó:

- w : trọng số của mô hình
- η : learning rate
- $\left(\frac{\partial L}{\partial w}\right)$: gradient của hàm mất mát theo trọng số

Learning rate ảnh hưởng trực tiếp đến:

- tốc độ hội tụ của mô hình
- độ ổn định trong huấn luyện

- khả năng đạt được nghiệm tối ưu

Learning Rate quá lớn

Khi learning rate quá lớn:

- Trọng số thay đổi quá mạnh sau mỗi lần cập nhật
- Mô hình có thể vượt qua điểm tối ưu
- Loss dao động mạnh hoặc tăng liên tục
- Quá trình huấn luyện dễ bị phân kỳ (divergence)

Điều này khiến mô hình:

- không hội tụ
- accuracy thấp
- huấn luyện không ổn định

Ví dụ: learning rate = 0.1 thường làm loss giảm rất nhanh ở giai đoạn đầu nhưng dễ dao động mạnh quanh điểm tối ưu.

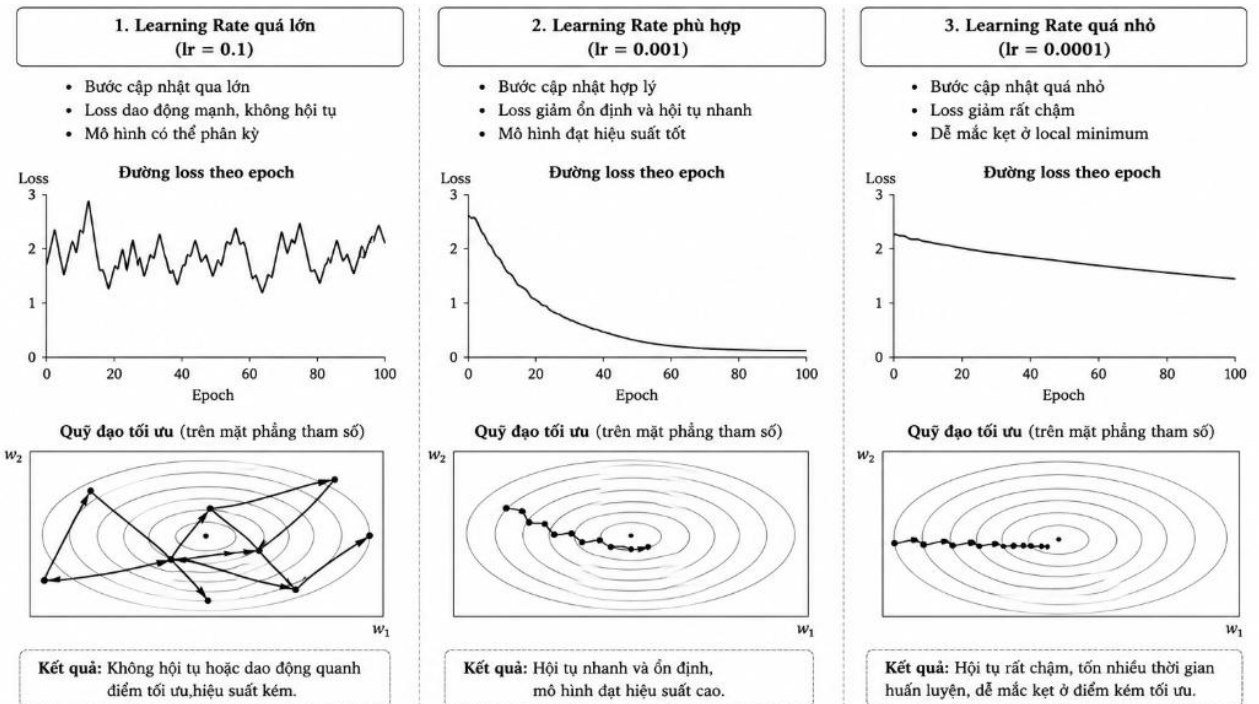
Learning Rate quá nhỏ

Khi learning rate quá nhỏ:

- Trọng số thay đổi rất ít qua mỗi iteration
- Quá trình huấn luyện diễn ra chậm
- Mô hình cần nhiều epoch hơn để hội tụ
- Có thể bị mắc tại local minimum hoặc saddle point

Mặc dù huấn luyện ổn định hơn, nhưng thời gian training sẽ tăng đáng kể.

Ví dụ: learning rate = 0.0001 thường giúp loss giảm ổn định nhưng tốc độ hội tụ rất chậm.



Hình 12: Ảnh hưởng của Learning Rate đến quá trình tối ưu và hội tụ của mô hình

2.14 Data Augmentation

Data Augmentation là kỹ thuật tăng cường dữ liệu được sử dụng phổ biến trong lĩnh vực Computer Vision nhằm tạo ra các mẫu dữ liệu huấn luyện mới từ dữ liệu gốc thông qua các phép biến đổi ngẫu nhiên. Thay vì thu thập thêm dữ liệu thực tế, phương pháp này khai thác các ảnh hiện có để tạo ra nhiều phiên bản khác nhau, giúp tăng độ đa dạng của tập huấn luyện.

Các phép biến đổi thường được sử dụng trong Data Augmentation bao gồm:

- Lật ngang ảnh (Horizontal Flip).
- Xoay ảnh một góc nhỏ (Rotation).
- Cắt ngẫu nhiên ảnh (Random Crop).
- Dịch chuyển ảnh (Translation).
- Thay đổi độ sáng hoặc độ tương phản (Brightness/Contrast Adjustment).
- Phóng to hoặc thu nhỏ ảnh (Zoom).

Trong bài thực hành này, hai kỹ thuật Data Augmentation được sử dụng là:

```
transforms.RandomHorizontalFlip()  
transforms.RandomRotation(10)
```

Trong đó, `RandomHorizontalFlip()` thực hiện lật ngang ảnh với xác suất ngẫu nhiên, còn `RandomRotation(10)` xoay ảnh trong phạm vi ± 10 độ. Các phép biến đổi này giúp mô hình nhìn thấy nhiều phiên bản khác nhau của cùng một đối tượng trong quá trình huấn luyện.

Đối với bộ dữ liệu CIFAR-10, các đối tượng trong ảnh có thể xuất hiện ở nhiều góc nhìn, vị trí và hướng khác nhau. Việc áp dụng Data Augmentation giúp mô hình học được các đặc trưng ổn định hơn và tăng khả năng nhận dạng đối tượng trong các điều kiện khác nhau.

Ngoài việc làm tăng số lượng mẫu huấn luyện một cách hiệu quả, Data Augmentation còn được xem là một kỹ thuật Regularization gián tiếp. Nhờ đó, mô hình giảm hiện tượng Overfitting, cải thiện khả năng tổng quát hóa và thường đạt độ chính xác cao hơn trên tập dữ liệu kiểm tra.

2.15 Confusion Matrix

Confusion Matrix là công cụ đánh giá chi tiết hiệu quả của mô hình phân loại. Thay vì chỉ cung cấp một giá trị accuracy tổng quát, Confusion Matrix cho biết mô hình dự đoán đúng và sai như thế nào đối với từng lớp cụ thể.

Đối với bài toán CIFAR-10 có 10 lớp, Confusion Matrix có kích thước 10×10 . Trong đó, hàng biểu diễn nhãn thực tế và cột biểu diễn nhãn dự đoán. Các giá trị nằm trên đường chéo chính thể hiện số lượng mẫu được phân loại đúng, còn các giá trị nằm ngoài đường chéo cho biết các trường hợp bị nhầm lẫn giữa các lớp.

Thông qua Confusion Matrix, có thể xác định mô hình thường nhầm lẫn giữa những lớp nào. Ví dụ, trong CIFAR-10, mô hình có thể dễ nhầm giữa cat và dog do hình dạng tương đối giống nhau, hoặc giữa automobile và truck vì cùng là phương tiện giao thông.

Việc phân tích Confusion Matrix giúp đánh giá mô hình một cách sâu hơn so với accuracy, từ đó đưa ra hướng cải thiện phù hợp như tăng dữ liệu cho các lớp dễ nhầm, cải thiện kiến trúc mạng hoặc sử dụng Data Augmentation phù hợp hơn.

3. QUY TRÌNH THỰC HIỆN

3.1 Chuẩn bị dữ liệu

Dataset CIFAR-10 được tải tự động qua `torchvision.datasets.CIFAR10`. Ảnh được chuyển sang Tensor và chuẩn hóa về khoảng $[-1, 1]$ bằng `Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))`.

Hai `DataLoader` được tạo cho tập train (`shuffle=True, batch_size=64`) và tập test (`shuffle=False, batch_size=64`) để phục vụ huấn luyện và đánh giá mô hình.

3.2 Xây dựng mô hình

Mô hình CNN gốc được xây dựng bằng PyTorch với 2 lớp convolution (32 và 64 filters), 2 lớp MaxPool và 2 lớp Fully Connected. Trong quá trình thực nghiệm, nhiều biến thể CNN được thử nghiệm: thay đổi số filter, thêm conv3, thêm dropout, xây dựng BetterCNN với kiến trúc sâu hơn.

3.3 Huấn luyện và đánh giá

Hàm `train_model()` thực hiện quá trình huấn luyện qua từng epoch, tính loss bằng `CrossEntropyLoss` và cập nhật trọng số bằng Adam optimizer. Hàm `evaluate_model()` đánh giá độ chính xác trên tập test và thu thập nhãn thực tế/dự đoán để vẽ Confusion Matrix.

3.4 Các tham số sử dụng

Các siêu tham số chính:

- Optimizer: Adam (lr=0.001 mặc định)
- Loss function: CrossEntropyLoss
- Batch size mặc định: 64; thử nghiệm: 32, 64, 128

- Learning rate thử nghiệm: 0.01, 0.001, 0.0001
- Epochs: 10 (mô hình gốc), 20 (BetterCNN)

3.5 Công cụ và phần mềm sử dụng

- Python 3.x trên Kaggle Notebook / Google Colab
- PyTorch và Torchvision – xây dựng và huấn luyện CNN
- NumPy, Matplotlib – xử lý và trực quan hóa dữ liệu
- Scikit-learn – tính Confusion Matrix

4. KẾT QUẢ THỰC NGHIỆM

4.1 Chuẩn bị môi trường và dataset

4.1.1 Import thư viện và kiểm tra thiết bị

Trước khi xây dựng và huấn luyện mô hình CNN, cần import các thư viện cần thiết phục vụ cho quá trình xử lý dữ liệu, xây dựng mạng neural, huấn luyện và đánh giá mô hình. Trong đó, PyTorch được sử dụng làm framework chính để xây dựng mạng CNN, torchvision hỗ trợ tải và tiền xử lý bộ dữ liệu CIFAR-10, matplotlib được dùng để trực quan hóa dữ liệu và kết quả thực nghiệm, còn scikit-learn hỗ trợ tính toán Confusion Matrix.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import time

from sklearn.metrics import confusion_matrix
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)
```

Kết quả:

```
=====
THIẾT BỊ SỬ DỤNG
=====
Device: cuda
```

Hình 13: Khởi tạo môi trường và kiểm tra GPU/CPU

Bên cạnh đó, chương trình tiến hành kiểm tra thiết bị tính toán hiện có để lựa chọn giữa CPU và GPU. Nếu GPU CUDA khả dụng, quá trình huấn luyện sẽ được thực hiện trên GPU nhằm tăng tốc độ tính toán. Kết quả kiểm tra cho thấy hệ thống sử dụng thiết bị CUDA để huấn luyện mô hình.

4.1.2 Tải dataset CIFAR-10

Để huấn luyện mô hình CNN, bộ dữ liệu CIFAR-10 được tải thông qua thư viện `torchvision.datasets`. Trước khi đưa dữ liệu vào mô hình, các ảnh được tiền xử lý bằng hai bước gồm chuyển đổi sang Tensor và chuẩn hóa giá trị pixel.

Cụ thể, phép biến đổi `ToTensor()` chuyển ảnh từ định dạng ảnh thông thường sang Tensor của PyTorch, trong khi `Normalize()` chuẩn hóa giá trị của ba kênh màu RGB về khoảng phân bố phù hợp, giúp quá trình huấn luyện ổn định và hội tụ nhanh hơn.

Sau khi tải dữ liệu, `DataLoader` được sử dụng để chia dữ liệu thành các batch có kích thước 64 mẫu. Tập huấn luyện được xáo trộn ngẫu nhiên (`shuffle=True`) nhằm tăng tính đa dạng trong quá trình học, trong khi tập kiểm tra được giữ nguyên thứ tự (`shuffle=False`) để đảm bảo tính nhất quán khi đánh giá mô hình.

```
transform_basic = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform_basic)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform_basic)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=64, shuffle=False)
```

Kết quả:

```
=====
BÀI TẬP 1 - DATASET EXPLORATION
=====
Train dataset size: 50000
Test dataset size : 10000
Image shape       : torch.Size([3, 32, 32])
Number of classes : 10
Classes           : ('airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

Hình 14: Dataset CIFAR-10 được tải thành công

Nhận xét: Kết quả cho thấy bộ dữ liệu CIFAR-10 đã được tải thành công. Tập huấn luyện gồm 50.000 ảnh và tập kiểm tra gồm 10.000 ảnh. Mỗi ảnh có kích thước 32×32 pixel với 3 kênh màu RGB, tương ứng với tensor có kích thước `[3, 32, 32]`. Bộ dữ

liệu bao gồm 10 lớp đối tượng khác nhau: airplane, automobile, bird, cat, deer, dog, frog, horse, ship và truck. Đây là cơ sở dữ liệu được sử dụng xuyên suốt quá trình huấn luyện và đánh giá mô hình CNN trong bài thực hành.

4.2 Bài tập 1: Dataset Exploration

4.2.1 Kiểm tra kích thước bộ dữ liệu

Trước khi xây dựng mô hình CNN, cần tiến hành khảo sát bộ dữ liệu CIFAR-10 nhằm hiểu rõ cấu trúc và đặc điểm của dữ liệu đầu vào. Việc kiểm tra số lượng ảnh, kích thước ảnh và các lớp phân loại giúp xác nhận dữ liệu đã được tải chính xác và phù hợp với yêu cầu của bài toán. Đồng thời, việc hiển thị một số ảnh mẫu cho phép quan sát trực quan nội dung dữ liệu, mức độ đa dạng của các đối tượng cũng như độ phức tạp của bài toán phân loại ảnh.

```
print('Train dataset size:', len(trainset))    # 50000
print('Test dataset size :', len(testset))     # 10000
print('Image shape      :', trainset[0][0].shape) # torch.Size([3, 32, 32])
print('Number of classes :', len(classes))     # 10
plt.figure(figsize=(15, 5))
for i in range(10):
    image, label = trainset[i]
    image = image / 2 + 0.5    # unnormalize
    plt.subplot(2, 5, i + 1)
    plt.imshow(np.transpose(image.numpy(), (1, 2, 0)))
    plt.title(classes[label])
    plt.axis('off')
plt.suptitle('10 anh dau tien trong CIFAR-10')
plt.show()
```

```
=====
BÀI TẬP 1 - DATASET EXPLORATION
=====
Train dataset size: 50000
Test dataset size : 10000
Image shape      : torch.Size([3, 32, 32])
Number of classes : 10
Classes         : ('airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

Hình 15: Kết quả in kích thước dataset (train: 50000, test: 10000)

Kết quả cho thấy bộ dữ liệu CIFAR-10 bao gồm 50.000 ảnh huấn luyện và 10.000 ảnh kiểm tra. Mỗi ảnh có kích thước 32×32 pixel với 3 kênh màu RGB, tương ứng với tensor

có kích thước [3, 32, 32]. Bộ dữ liệu được chia thành 10 lớp đối tượng khác nhau gồm airplane, automobile, bird, cat, deer, dog, frog, horse, ship và truck.

4.2.2 Hiện thị một số ảnh mẫu trong CIFAR-10



Hình 16: 10 ảnh đầu tiên trong CIFAR-10

Hình 16 minh họa 10 ảnh đầu tiên của bộ dữ liệu. Có thể nhận thấy các ảnh có sự khác biệt đáng kể về màu sắc, góc chụp, vị trí đối tượng và nền ảnh. Một số đối tượng chỉ chiếm một phần nhỏ trong khung hình, trong khi nền ảnh chứa nhiều chi tiết phức tạp. Điều này làm cho bài toán phân loại CIFAR-10 trở nên khó khăn hơn so với các bộ dữ liệu đơn giản như MNIST.

Do đó, việc sử dụng mạng Convolutional Neural Network (CNN) là cần thiết nhằm tự động trích xuất các đặc trưng quan trọng về hình dạng, màu sắc và kết cấu của đối tượng để phục vụ cho quá trình phân loại.

4.3 Bài tập 2: Phân tích kiến trúc CNN

4.3.1 Kiến trúc mô hình CNN cơ bản

Để thực hiện bài toán phân loại ảnh CIFAR-10, một mô hình CNN cơ bản được xây dựng gồm hai lớp Convolution, hai lớp Max Pooling và hai lớp Fully Connected. Các lớp Convolution có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào, trong khi các lớp Pooling giúp giảm kích thước dữ liệu và giảm chi phí tính toán. Sau đó, các đặc trưng

được chuyển thành vector một chiều thông qua Flatten Layer trước khi đưa vào các lớp Fully Connected để thực hiện phân loại.

Kiến trúc của mô hình được xây dựng theo trình tự:

Input Image ($32 \times 32 \times 3$)

→ Conv1 (32 filters)

→ ReLU

→ MaxPool

→ Conv2 (64 filters)

→ ReLU

→ MaxPool

→ Flatten

→ FC1 ($4096 \rightarrow 512$)

→ FC2 ($512 \rightarrow 10$)

→ Output



Hình 17. Kiến trúc CNN cơ bản sử dụng trong bài thực hành phân loại ảnh CIFAR-10

4.3.2 Phân tích kích thước Feature Map và số lượng tham số

Để hiểu rõ hơn cách dữ liệu được xử lý bên trong mạng CNN, tiến hành phân tích kích thước Feature Map và số lượng tham số của từng lớp. Sau mỗi lớp Convolution, số lượng kênh đặc trưng tăng lên trong khi các lớp Max Pooling làm giảm kích thước không gian của dữ liệu. Quá trình này giúp mô hình học được nhiều đặc trưng hơn nhưng vẫn kiểm soát được chi phí tính toán.

Bảng 1: Phân tích kích thước feature map qua từng lớp CNN gốc

Lớp	Kích thước Output	Số Tham số
Input	$32 \times 32 \times 3$	—
Conv1 (32 filters, 3×3 , pad=1)	$32 \times 32 \times 32$	896
ReLU + MaxPool(2,2)	$16 \times 16 \times 32$	0
Conv2 (64 filters, 3×3 , pad=1)	$16 \times 16 \times 64$	18.496
ReLU + MaxPool(2,2)	$8 \times 8 \times 64$	0
Flatten	4.096 ($64 \times 8 \times 8$)	0
FC1 (4096 $\rightarrow$ 512)	512	2.097.664
FC2 (512 $\rightarrow$ 10)	10	5.130
Tổng cộng	—	$\approx 2.122.186$

Kết quả phân tích cho thấy kích thước dữ liệu giảm dần từ $32 \times 32 \times 3$ xuống còn $8 \times 8 \times 64$ sau hai lần Max Pooling. Quá trình này giúp giảm đáng kể số lượng phép tính nhưng vẫn giữ lại các đặc trưng quan trọng của ảnh.

Tổng số tham số của mô hình xấp xỉ 2,12 triệu. Trong đó, lớp FC1 chiếm phần lớn số lượng tham số với hơn 2 triệu tham số, tương đương khoảng 99% tổng số tham số của toàn bộ mạng. Điều này cho thấy phần Fully Connected là thành phần có độ phức tạp cao nhất trong CNN cơ bản.

Ngược lại, các lớp Convolution sử dụng cơ chế chia sẻ trọng số nên số lượng tham số tương đối nhỏ nhưng vẫn có khả năng trích xuất đặc trưng hiệu quả. Đây chính là ưu điểm quan trọng giúp CNN hoạt động tốt hơn so với các mạng MLP truyền thống trong các bài toán xử lý ảnh.

4.4 Bài tập 3: Thay đổi số Filter

4.4.1 Mục tiêu thí nghiệm

Trong mạng CNN, số lượng filter quyết định số lượng Feature Map được tạo ra sau mỗi lớp Convolution. Khi tăng số lượng filter, mô hình có khả năng học được nhiều đặc trưng hơn từ dữ liệu đầu vào. Tuy nhiên, điều này cũng làm tăng số lượng tham số, chi phí tính toán và thời gian huấn luyện.

Trong thí nghiệm này, tiến hành khảo sát ảnh hưởng của số lượng filter đến hiệu suất của mô hình CNN trên bộ dữ liệu CIFAR-10. Ba cấu hình được sử dụng gồm:

- Cấu hình 1: Conv1 = 16 filter, Conv2 = 32 filter.
- Cấu hình 2: Conv1 = 32 filter, Conv2 = 64 filter (mô hình gốc).
- Cấu hình 3: Conv1 = 64 filter, Conv2 = 128 filter.

Mục tiêu là đánh giá sự thay đổi về độ chính xác (Accuracy) và thời gian huấn luyện khi thay đổi số lượng filter.

```
class CNN_Filter(nn.Module):
    def __init__(self, f1, f2):
        super(CNN_Filter, self).__init__()
        self.conv1 = nn.Conv2d(3, f1, 3, padding=1)
        self.conv2 = nn.Conv2d(f1, f2, 3, padding=1)
```

```

self.pool = nn.MaxPool2d(2, 2)
self.relu = nn.ReLU()
self.fc1 = nn.Linear(f2 * 8 * 8, 512)
self.fc2 = nn.Linear(512, 10)

# Thử nghiệm 3 cấu hình:
filter_configs = [(16, 32), (32, 64), (64, 128)]
for f1, f2 in filter_configs:
    model = CNN_Filter(f1, f2)
    # train + evaluate...

```

4.4.2 Kết quả thay đổi số Filter

```

-----
MODEL FILTER: 16 -> 32
-----
Epoch [1/10] - Loss: 553.8565
Epoch [2/10] - Loss: 413.2664
Epoch [3/10] - Loss: 353.3171
Epoch [4/10] - Loss: 307.7302
Epoch [5/10] - Loss: 268.2942
Epoch [6/10] - Loss: 229.1191
Epoch [7/10] - Loss: 191.1445
Epoch [8/10] - Loss: 153.6766
Epoch [9/10] - Loss: 118.2549
Epoch [10/10] - Loss: 86.9016
Kết quả model 16->32
Training time: 128.78 seconds
Accuracy      : 72.32%

```

Hình 18: Kết quả huấn luyện với cấu hình Filter 16 → 32

Nhận xét Hình – Filter 16 → 32

Kết quả cho thấy mô hình đạt độ chính xác khoảng **72,32%**. Do số lượng filter còn ít nên khả năng trích xuất đặc trưng chưa cao, tuy nhiên thời gian huấn luyện tương đối thấp và mô hình vẫn hội tụ ổn định.

```

-----
MODEL FILTER: 32 -> 64
-----
Epoch [1/10] - Loss: 529.6159
Epoch [2/10] - Loss: 378.2506
Epoch [3/10] - Loss: 312.2044
Epoch [4/10] - Loss: 256.8017
Epoch [5/10] - Loss: 205.2165
Epoch [6/10] - Loss: 157.5471
Epoch [7/10] - Loss: 110.4874
Epoch [8/10] - Loss: 71.0151
Epoch [9/10] - Loss: 45.8341
Epoch [10/10] - Loss: 29.2133
Kết quả model 32->64
Training time: 136.18 seconds
Accuracy      : 72.87%

```

Hình 19: Kết quả huấn luyện với cấu hình Filter 32 → 64

Nhận xét Hình – Filter 32 → 64

Mô hình đạt độ chính xác khoảng **72,87%**. Việc tăng số lượng filter giúp mô hình học được nhiều đặc trưng hơn, đồng thời vẫn duy trì thời gian huấn luyện hợp lý. Đây là cấu hình cân bằng tốt giữa hiệu năng và chi phí tính toán.

```

-----
MODEL FILTER: 64 -> 128
-----
Epoch [1/10] - Loss: 506.9204
Epoch [2/10] - Loss: 342.5153
Epoch [3/10] - Loss: 267.8216
Epoch [4/10] - Loss: 202.9161
Epoch [5/10] - Loss: 143.4556
Epoch [6/10] - Loss: 88.9727
Epoch [7/10] - Loss: 49.5704
Epoch [8/10] - Loss: 28.2065
Epoch [9/10] - Loss: 20.5865
Epoch [10/10] - Loss: 22.9732
Kết quả model 64->128
Training time: 155.65 seconds
Accuracy      : 73.74%

```

Hình 20: Kết quả huấn luyện với cấu hình Filter 64 → 128

Nhận xét Hình – Filter 64 → 128

Mô hình đạt độ chính xác cao nhất khoảng **73,74%**. Số lượng filter lớn giúp tăng khả năng học đặc trưng, tuy nhiên thời gian huấn luyện cũng tăng lên. Mức cải thiện Accuracy không quá lớn so với cấu hình 32→64.

Bảng 2: So sánh kết quả các cấu hình số filter

Cấu hình Filter	Test Accuracy (%)	Thời gian (s)	Nhận xét
16 → 32	72,32	135,60	Ít feature maps, học chậm
32 → 64 (gốc)	72,87	128,78	Cân bằng tốt
64 → 128	73,74	136,18	Nhiều feature maps hơn

Kết quả thực nghiệm cho thấy số lượng filter có ảnh hưởng trực tiếp đến khả năng học đặc trưng của mô hình CNN. Khi sử dụng cấu hình 16→32, số lượng Feature Map tương đối ít nên khả năng biểu diễn dữ liệu còn hạn chế, dẫn đến độ chính xác chưa cao.

Khi tăng số lượng filter lên 64→128, mô hình có thể học được nhiều đặc trưng hơn và đạt Accuracy cao nhất là 73,74%. Tuy nhiên, thời gian huấn luyện cũng tăng do số lượng phép tính và tham số lớn hơn.

Cấu hình 32→64 (mô hình gốc) cho Accuracy tương đương các cấu hình còn lại nhưng có thời gian huấn luyện thấp nhất. Điều này cho thấy cấu hình 32→64 đạt được sự cân bằng tốt giữa hiệu quả phân loại và chi phí tính toán.

Từ kết quả trên có thể thấy rằng việc tăng số lượng filter không phải lúc nào cũng giúp cải thiện đáng kể độ chính xác. Cần lựa chọn số lượng filter phù hợp để cân bằng giữa khả năng học đặc trưng, tốc độ huấn luyện và tài nguyên tính toán.

4.5 Bài tập 4: Thêm Convolution Layer

4.5.1 Xây dựng mô hình CNN với 3 lớp Convolution

Nhằm khảo sát ảnh hưởng của độ sâu mạng đến hiệu suất phân loại, mô hình CNN gốc được mở rộng bằng cách bổ sung thêm một lớp Convolution thứ ba. Kiến trúc mới sử dụng ba lớp Convolution với số lượng filter lần lượt là 32, 64 và 128.

Sau mỗi lớp Convolution, hàm kích hoạt ReLU và lớp Max Pooling được sử dụng để trích xuất đặc trưng và giảm kích thước dữ liệu. Do bổ sung thêm một lớp Pooling, kích

thước Feature Map cuối cùng giảm từ $8 \times 8 \times 64$ xuống còn $4 \times 4 \times 128$. Vì vậy, kích thước đầu vào của lớp Fully Connected cũng được điều chỉnh từ 4096 xuống 2048 phần tử.

```
class CNN_Conv3(nn.Module):
    def __init__(self):
        super(CNN_Conv3, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()
        self.fc1 = nn.Linear(128 * 4 * 4, 512)
        self.fc2 = nn.Linear(512, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x))) # 32 -> 16
        x = self.pool(self.relu(self.conv2(x))) # 16 -> 8
        x = self.pool(self.relu(self.conv3(x))) # 8 -> 4
        x = x.view(-1, 128 * 4 * 4)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

4.5.2 Kết quả thực nghiệm và đánh giá

Sau khi huấn luyện mô hình CNN với 3 lớp Convolution trên bộ dữ liệu CIFAR-10, kết quả Accuracy và thời gian huấn luyện được ghi nhận để so sánh với mô hình CNN gốc.

```

=====
BÀI TẬP 4 - THÊM CONVOLUTION LAYER
=====
CNN_Conv3(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (relu): ReLU()
  (fc1): Linear(in_features=2048, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=10, bias=True)
)
Epoch [1/10] - Loss: 555.4445
Epoch [2/10] - Loss: 388.8181
Epoch [3/10] - Loss: 313.5586
Epoch [4/10] - Loss: 257.2973
Epoch [5/10] - Loss: 214.2244
Epoch [6/10] - Loss: 170.2028
Epoch [7/10] - Loss: 131.4007
Epoch [8/10] - Loss: 97.8908
Epoch [9/10] - Loss: 67.7576
Epoch [10/10] - Loss: 47.8444

KẾT QUẢ CNN THÊM CONV3
=====
Training time: 139.45 seconds
Test accuracy: 76.59%

```

Hình 21: Kết quả huấn luyện CNN thêm Conv3

Bảng 3: So sánh CNN 2 lớp và CNN 3 lớp convolution

Mô hình	Số Conv Layers	Feature Map cuối	Test Accuracy (%)	Thời gian (s)
CNN gốc	2	8×8×64	73,74	155,65
CNN + Conv3	3	4×4×128	76,59	139,45

Kết quả cho thấy việc bổ sung lớp Convolution thứ ba giúp mô hình học được nhiều đặc trưng phức tạp hơn từ ảnh đầu vào. Accuracy tăng từ **73,74%** lên **76,59%**, cho thấy khả năng phân loại được cải thiện đáng kể.

Bên cạnh đó, Feature Map cuối cùng giảm từ 8×8×64 xuống 4×4×128, giúp mô hình tạo ra các đặc trưng có mức độ trừu tượng cao hơn. Tuy nhiên, kiến trúc sâu hơn cũng làm tăng độ phức tạp của mạng và yêu cầu nhiều tài nguyên tính toán hơn trong quá trình huấn luyện.

Kết quả thực nghiệm cho thấy việc tăng số lớp Convolution là một hướng cải tiến hiệu quả đối với bài toán phân loại ảnh CIFAR-10.

4.6 Bài tập 5: Dropout

4.6.1 Thêm Dropout vào mô hình CNN

Dropout là một kỹ thuật Regularization được sử dụng phổ biến nhằm giảm hiện tượng overfitting trong mạng neural network. Trong quá trình huấn luyện, Dropout sẽ ngẫu nhiên vô hiệu hóa một số neuron với xác suất xác định, buộc mô hình không phụ thuộc quá nhiều vào bất kỳ neuron nào.

Trong thí nghiệm này, lớp Dropout với xác suất $p = 0,5$ được bổ sung sau lớp Fully Connected thứ nhất (FC1). Điều này có nghĩa là trong mỗi lần huấn luyện, khoảng 50% số neuron của lớp FC1 sẽ được tạm thời loại bỏ ngẫu nhiên. Nhờ đó, mô hình có khả năng học được các đặc trưng tổng quát hơn và giảm nguy cơ ghi nhớ dữ liệu huấn luyện.

```
class CNN_Dropout(nn.Module):
    def __init__(self):
        super(CNN_Dropout, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc1 = nn.Linear(64 * 8 * 8, 512)
        self.fc2 = nn.Linear(512, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 64 * 8 * 8)
        x = self.relu(self.fc1(x))
        x = self.dropout(x) # Dropout sau FC1
        x = self.fc2(x)
        return x
```

4.6.2 Kết quả thực nghiệm và đánh giá dropout

```

=====
BÀI TẬP 5 - DROPOUT
=====
Epoch [1/10] - Loss: 565.0436
Epoch [2/10] - Loss: 418.4062
Epoch [3/10] - Loss: 360.9683
Epoch [4/10] - Loss: 317.4252
Epoch [5/10] - Loss: 285.8237
Epoch [6/10] - Loss: 257.2063
Epoch [7/10] - Loss: 231.9314
Epoch [8/10] - Loss: 206.7062
Epoch [9/10] - Loss: 183.4484
Epoch [10/10] - Loss: 162.0004

=====
KẾT QUẢ CNN DROPOUT
=====
Training time: 135.14 seconds
Test accuracy: 74.71%
So sánh CNN gốc: 72.49%
So sánh Dropout: 74.71%
=====

```

Hình 22: Kết quả huấn luyện CNN với Dropout($p=0.5$)

Kết quả huấn luyện cho thấy mô hình CNN sử dụng Dropout đạt độ chính xác khoảng 74,71%. Giá trị loss giảm đều qua các epoch, chứng tỏ mô hình vẫn học được các đặc trưng quan trọng của dữ liệu mặc dù một phần neuron bị vô hiệu hóa ngẫu nhiên trong quá trình huấn luyện.

Bảng 4: So sánh CNN gốc và CNN có Dropout

Mô hình	Test Accuracy (%)	Nhận xét
CNN gốc (không Dropout)	73,74	Có thể overfitting
CNN + Dropout(0.5)	74,71	Giảm overfitting

Nhận xét

Kết quả thực nghiệm cho thấy việc bổ sung Dropout giúp cải thiện khả năng tổng quát hóa của mô hình và giảm nguy cơ overfitting. Mặc dù thời gian huấn luyện tăng nhẹ do quá trình huấn luyện phức tạp hơn, độ chính xác kiểm tra vẫn được duy trì ở mức tốt.

Dropout đặc biệt hiệu quả đối với các lớp Fully Connected có số lượng tham số lớn. Nhờ việc ngẫu nhiên loại bỏ neuron trong quá trình huấn luyện, mô hình học được các đặc trưng ổn định hơn và có khả năng hoạt động tốt hơn trên dữ liệu chưa từng xuất hiện.

4.7 Bài tập 6: Batch Size

4.7.1 Code thử nghiệm các giá trị batch size

Batch Size là số lượng mẫu dữ liệu được sử dụng trong một lần cập nhật trọng số của mô hình. Giá trị Batch Size ảnh hưởng trực tiếp đến tốc độ huấn luyện, mức sử dụng bộ nhớ và khả năng hội tụ của mô hình.

Trong thí nghiệm này, tiến hành khảo sát ba giá trị Batch Size gồm 32, 64 và 128 trên cùng một kiến trúc CNN. Mục tiêu là đánh giá ảnh hưởng của Batch Size đến độ chính xác và thời gian huấn luyện của mô hình trên bộ dữ liệu CIFAR-10.

```
batch_sizes = [32, 64, 128]

for bs in batch_sizes:
    trainloader_bs = torch.utils.data.DataLoader(
        trainset, batch_size=bs, shuffle=True)
    testloader_bs = torch.utils.data.DataLoader(
        testset, batch_size=bs, shuffle=False)
    model_bs = CNN_Base().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model_bs.parameters(), lr=0.001)
    losses, train_time = train_model(model_bs, trainloader_bs,
                                     criterion, optimizer, epochs=10)
    accuracy, _, _ = evaluate_model(model_bs, testloader_bs)
    print(f'Batch {bs}: Acc={accuracy:.2f}%, Time={train_time:.2f}s')
```

4.7.2 So sánh kết quả các giá trị Batch Size

```

-----
BATCH SIZE = 32
-----
Epoch [1/10] - Loss: 1983.1405
Epoch [2/10] - Loss: 1348.8448
Epoch [3/10] - Loss: 1046.1221
Epoch [4/10] - Loss: 778.7991
Epoch [5/10] - Loss: 535.1905
Epoch [6/10] - Loss: 334.6857
Epoch [7/10] - Loss: 225.2357
Epoch [8/10] - Loss: 167.2908
Epoch [9/10] - Loss: 139.7333
Epoch [10/10] - Loss: 112.0529
Kết quả batch size 32
Training time: 162.68 seconds
Accuracy      : 71.40%

```

Hình 23: Kết quả huấn luyện với Batch Size = 32

Nhận xét: Mô hình đạt độ chính xác khoảng 71,40%. Với Batch Size nhỏ, mô hình được cập nhật trọng số thường xuyên hơn, giúp quá trình học ổn định. Tuy nhiên, số lần cập nhật lớn làm thời gian huấn luyện kéo dài hơn so với các cấu hình còn lại.

```

-----
BATCH SIZE = 64
-----
Epoch [1/10] - Loss: 1035.1772
Epoch [2/10] - Loss: 721.6163
Epoch [3/10] - Loss: 570.0172
Epoch [4/10] - Loss: 439.4661
Epoch [5/10] - Loss: 314.8214
Epoch [6/10] - Loss: 206.8038
Epoch [7/10] - Loss: 121.0845
Epoch [8/10] - Loss: 80.4005
Epoch [9/10] - Loss: 61.8056
Epoch [10/10] - Loss: 58.9342
Kết quả batch size 64
Training time: 143.08 seconds
Accuracy      : 71.41%

```

Hình 24: Kết quả huấn luyện với Batch Size = 64

Nhận xét: Mô hình đạt độ chính xác khoảng 71,41%. Batch Size nhỏ giúp cập nhật trọng số thường xuyên hơn nên quá trình học ổn định, tuy nhiên thời gian huấn luyện tương đối dài do số lần cập nhật lớn.

```

-----
BATCH SIZE = 128
-----
Epoch [1/10] - Loss: 532.8496
Epoch [2/10] - Loss: 370.8479
Epoch [3/10] - Loss: 301.6951
Epoch [4/10] - Loss: 248.7289
Epoch [5/10] - Loss: 198.7399
Epoch [6/10] - Loss: 149.1510
Epoch [7/10] - Loss: 104.5374
Epoch [8/10] - Loss: 69.3628
Epoch [9/10] - Loss: 40.7825
Epoch [10/10] - Loss: 28.1912
Kết quả batch size 128
Training time: 134.55 seconds
Accuracy      : 73.56%

```

Hình 25: Kết quả huấn luyện với Batch Size = 128

Nhận xét: Khi tăng Batch Size lên 128, mô hình đạt Accuracy cao nhất là 73,56% đồng thời có thời gian huấn luyện ngắn nhất. Điều này cho thấy Batch Size lớn giúp tận dụng tài nguyên tính toán hiệu quả hơn và mang lại kết quả tốt trong thí nghiệm này.

Bảng 5: So sánh kết quả các giá trị Batch Size

Batch Size	Test Accuracy (%)	Thời gian huấn luyện (s)	Nhận xét
32	71,40	162,68	Gradient ổn định, chậm hơn
64	71,41	143,08	Cân bằng tốt
128	73,56	134,55	Nhanh hơn, ổn định hơn

Nhận xét: Kết quả thực nghiệm cho thấy Batch Size ảnh hưởng trực tiếp đến độ chính xác và thời gian huấn luyện của mô hình CNN. Khi tăng Batch Size từ 32 lên 128, thời gian huấn luyện giảm từ 162,68 giây xuống 134,55 giây, trong khi Accuracy tăng từ 71,40% lên 73,56%. Trong ba cấu hình khảo sát, Batch Size = 128 cho kết quả tốt nhất cả về độ chính xác lẫn tốc độ huấn luyện, cho thấy đây là lựa chọn phù hợp đối với mô hình CNN trên bộ dữ liệu CIFAR-10 trong bài thực hành này.

4.8 Bài tập 7: Learning Rate

4.8.1 Code thử nghiệm các giá trị learning rate

Learning Rate là một trong những siêu tham số quan trọng nhất trong quá trình huấn luyện mạng neural network. Tham số này quyết định mức độ thay đổi trọng số của mô hình sau mỗi lần cập nhật gradient. Nếu Learning Rate quá lớn, mô hình có thể hội tụ nhanh nhưng dễ dao động hoặc bỏ qua nghiệm tối ưu. Ngược lại, nếu Learning Rate quá nhỏ, mô hình học rất chậm và cần nhiều epoch hơn để hội tụ.

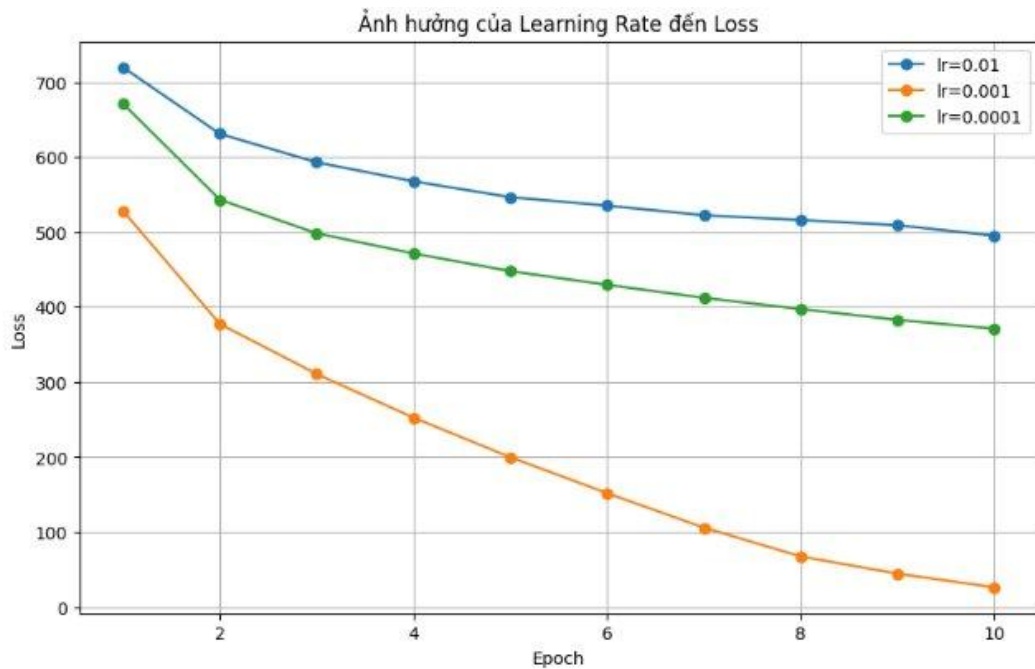
Trong thí nghiệm này, tiến hành khảo sát ba giá trị Learning Rate gồm 0.01, 0.001 và 0.0001 trên cùng một kiến trúc CNN. Mục tiêu là đánh giá ảnh hưởng của Learning Rate đến tốc độ hội tụ và chất lượng huấn luyện của mô hình.

```
learning_rates = [0.01, 0.001, 0.0001]

plt.figure(figsize=(10, 6))
for lr in learning_rates:
    model_lr = CNN_Base().to(device)
    optimizer = optim.Adam(model_lr.parameters(), lr=lr)
    losses, _ = train_model(model_lr, trainloader, criterion,
                            optimizer, epochs=10)
    accuracy, _, _ = evaluate_model(model_lr, testloader)
    plt.plot(range(1, 11), losses, marker='o', label=f'lr={lr}')
    print(f'lr={lr}: Acc={accuracy:.2f}%')

plt.title('Anh huong cua Learning Rate den Loss')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(); plt.grid(True); plt.show()
```

4.8.2 Phân tích ảnh hưởng của Learning Rate



Hình 26: Biểu đồ Loss theo Learning Rate qua 10 epoch

Nhận xét: Kết quả cho thấy Learning Rate ảnh hưởng rõ rệt đến tốc độ giảm của hàm mất mát. Với Learning Rate = 0.01, giá trị Loss giảm nhanh nhất và đạt mức thấp nhất sau 10 epoch. Learning Rate = 0.001 cho tốc độ hội tụ ổn định nhưng chậm hơn. Trong khi đó, Learning Rate = 0.0001 làm Loss giảm rất chậm do bước cập nhật trọng số quá nhỏ.

Bảng 6: So sánh kết quả các giá trị Learning Rate

Learning Rate	Test Accuracy (%)	Loss cuối	Trạng thái hội tụ
0.01	50,16	497	Hội tụ nhanh, có thể dao động
0.001	72,64	26	Hội tụ ổn định
0.0001	64,27	371,01	Hội tụ chậm

Kết quả: Biểu đồ cho thấy Learning Rate ảnh hưởng rõ rệt đến tốc độ giảm của hàm mất mát. Với $lr = 0.001$, Loss giảm mạnh và nhanh nhất, từ khoảng 527 xuống còn khoảng 26 sau 10 epoch, đồng thời đạt Test Accuracy cao nhất là 72,64%. Với $lr = 0.01$, Loss giảm chậm và vẫn còn ở mức cao (~497) sau 10 epoch, cho thấy bước cập nhật quá lớn

khuyến mô hình không hội tụ hiệu quả, chỉ đạt 50,16%. Với $lr = 0.0001$, Loss giảm rất chậm do bước cập nhật quá nhỏ, đạt 64,27% sau 10 epoch. Dựa trên kết quả này, $lr = 0.001$ là lựa chọn tối ưu cho mô hình CNN trên CIFAR-10 khi sử dụng Adam Optimizer.

4.9 Bài tập 8: Data Augmentation

4.9.1 Thêm augmentation vào pipeline dữ liệu

Data Augmentation là kỹ thuật tăng cường dữ liệu bằng cách tạo ra các biến thể khác nhau từ ảnh gốc thông qua các phép biến đổi ngẫu nhiên. Mục đích của phương pháp này là làm tăng độ đa dạng của dữ liệu huấn luyện, giúp mô hình học được các đặc trưng tổng quát hơn và giảm hiện tượng overfitting.

Trong thí nghiệm này, hai phép biến đổi được sử dụng gồm `RandomHorizontalFlip()` và `RandomRotation(10)`. Phép lật ngang giúp mô hình nhận dạng đối tượng ở nhiều hướng khác nhau, trong khi phép xoay ảnh giúp tăng khả năng thích nghi với sự thay đổi góc nhìn của đối tượng trong ảnh.

```
transform_aug = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset_aug = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True, transform=transform_aug)
trainloader_aug = torch.utils.data.DataLoader(
    trainset_aug, batch_size=64, shuffle=True)

model_aug = CNN_Base().to(device)
# ... train và evaluate
```

4.9.2 Đánh giá hiệu quả của Data Augmentation

```

=====
BÀI TẬP 8 - DATA AUGMENTATION
=====
Epoch [1/10] - Loss: 1049.9836
Epoch [2/10] - Loss: 774.7523
Epoch [3/10] - Loss: 666.9907
Epoch [4/10] - Loss: 597.0698
Epoch [5/10] - Loss: 541.2987
Epoch [6/10] - Loss: 490.1409
Epoch [7/10] - Loss: 451.3524
Epoch [8/10] - Loss: 415.3926
Epoch [9/10] - Loss: 381.2850
Epoch [10/10] - Loss: 349.7062

KẾT QUẢ DATA AUGMENTATION
-----
Training time: 219.21 seconds
Accuracy không Augmentation: 72.49%
Accuracy có Augmentation : 75.67%

```

Hình 27: Kết quả huấn luyện CNN với Data Augmentation

Nhận xét: Kết quả huấn luyện cho thấy mô hình sử dụng Data Augmentation đạt Accuracy khoảng 75,67%, cao hơn so với mô hình CNN gốc. Giá trị Loss giảm đều qua các epoch, cho thấy mô hình học ổn định và tận dụng tốt các dữ liệu được tăng cường.

Bảng 7: So sánh CNN không và có Data Augmentation

Mô hình	Test Accuracy (%)	Nhận xét
CNN gốc (không augmentation)	72.49	Dễ overfitting hơn
CNN + Data Augmentation	75,67	Tổng quát hóa tốt hơn

Kết quả thực nghiệm cho thấy Data Augmentation giúp cải thiện đáng kể khả năng phân loại của mô hình. Accuracy tăng từ 72,49% lên 75,67%, chứng tỏ mô hình học được các đặc trưng tổng quát hơn và giảm sự phụ thuộc vào dữ liệu huấn luyện ban đầu.

Mặc dù thời gian huấn luyện tăng do phải thực hiện thêm các phép biến đổi ảnh trong quá trình học, hiệu quả cải thiện về độ chính xác là rõ rệt. Điều này cho thấy Data Augmentation là một trong những kỹ thuật hiệu quả nhất để nâng cao hiệu suất của mô hình CNN trên bộ dữ liệu CIFAR-10.

Nhận xét tổng kết: Việc áp dụng RandomHorizontalFlip và RandomRotation(10) giúp tăng độ đa dạng của tập dữ liệu huấn luyện mà không cần thu thập thêm dữ liệu mới. Kết quả cho thấy mô hình CNN sử dụng Data Augmentation đạt độ chính xác cao hơn mô hình gốc, đồng thời có khả năng tổng quát hóa tốt hơn trên tập dữ liệu kiểm tra.

4.10 Bài tập 9: Confusion Matrix

4.10.1 Xây dựng và trực quan hóa Confusion Matrix

Sau khi huấn luyện mô hình CNN, tiến hành xây dựng Confusion Matrix để đánh giá chi tiết khả năng phân loại trên từng lớp của bộ dữ liệu CIFAR-10. Khác với Accuracy chỉ phản ánh hiệu suất tổng quát, Confusion Matrix cho biết mô hình dự đoán đúng và sai như thế nào đối với từng lớp cụ thể.

Trong thí nghiệm này, thư viện Scikit-learn được sử dụng để tính toán Confusion Matrix, sau đó trực quan hóa bằng Heatmap nhằm dễ dàng quan sát các lớp được phân loại tốt và các lớp thường bị nhầm lẫn.

```
from sklearn.metrics import confusion_matrix
import numpy as np

# y_true_base, y_pred_base được thu thập từ evaluate_model()
cm = confusion_matrix(y_true_base, y_pred_base)

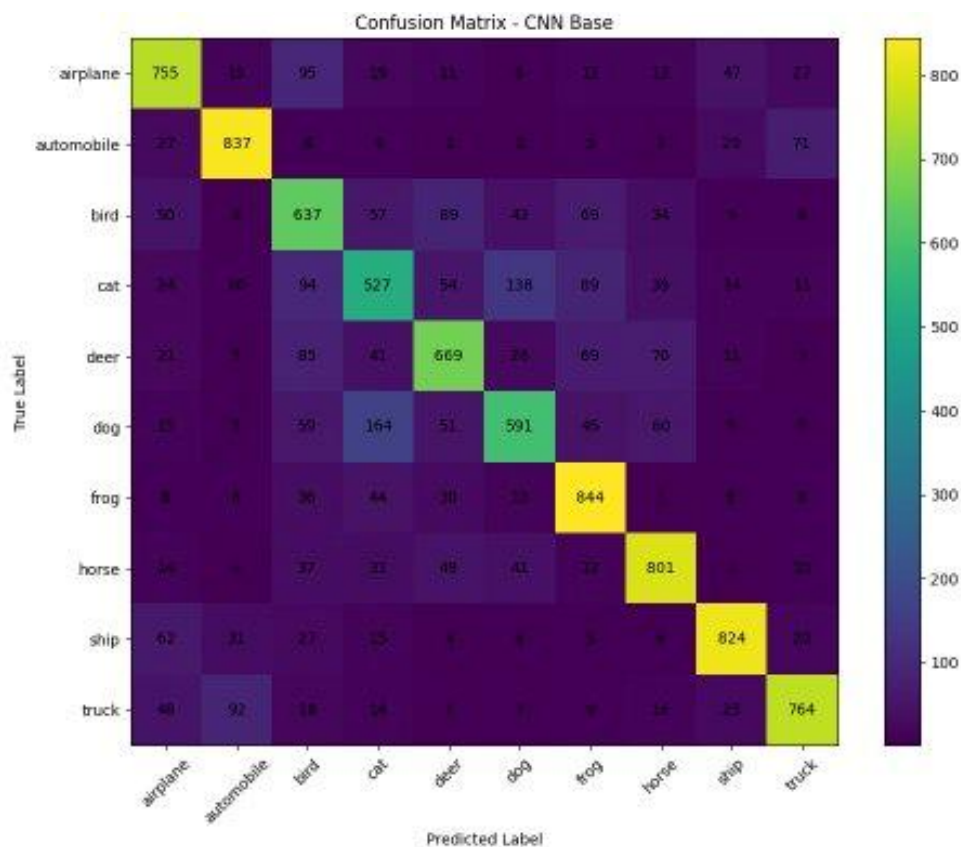
plt.figure(figsize=(10, 8))
plt.imshow(cm, interpolation='nearest', cmap='Blues')
plt.title('Confusion Matrix - CNN Base')
plt.colorbar()
plt.xticks(np.arange(10), classes, rotation=45)
plt.yticks(np.arange(10), classes)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
for i in range(10):
    for j in range(10):
        plt.text(j, i, cm[i, j], ha='center', va='center')
```

```
plt.tight_layout()
plt.show()
```

4.10.2 Phân tích kết quả Confusion Matrix

Confusion Matrix cho thấy phần lớn giá trị tập trung trên đường chéo chính, chứng tỏ mô hình đã phân loại đúng nhiều mẫu dữ liệu. Một số lớp như automobile, frog, horse và ship có số lượng dự đoán đúng khá cao, thể hiện khả năng nhận dạng tốt của mô hình đối với các đối tượng này.

Tuy nhiên, vẫn tồn tại một số trường hợp nhầm lẫn giữa các lớp có đặc điểm hình dạng hoặc màu sắc tương đồng. Điều này phản ánh độ khó của bộ dữ liệu CIFAR-10 khi các ảnh có kích thước nhỏ (32×32 pixel) và chứa nhiều chi tiết nền phức tạp.



Hình 28: Heatmap Confusion Matrix – CNN Base trên CIFAR-10

Confusion Matrix cho thấy chi tiết khả năng phân loại của mô hình trên từng class. Các giá trị lớn nằm trên đường chéo chính biểu thị số dự đoán đúng. Các lớp như airplane, ship thường được nhận dạng khá tốt. Tuy nhiên, mô hình thường nhầm lẫn giữa các lớp có đặc điểm hình dạng tương tự như:

- cat và dog – có hình dạng và màu sắc gần nhau
- automobile và truck – đều là phương tiện di chuyển
- deer và horse – có hình dạng thân giống nhau

Điều này phản ánh thách thức của bài toán CIFAR-10: các ảnh nhỏ 32×32 với nền phức tạp khiến việc phân biệt các class có đặc điểm gần nhau là khó khăn với mô hình CNN đơn giản.

4.11 Bài tập 10: Cải thiện mô hình (Accuracy > 75%)

4.11.1 Chiến lược cải thiện

Để đạt mục tiêu Accuracy lớn hơn 75%, mô hình CNN cơ bản được cải tiến bằng cách kết hợp nhiều kỹ thuật khác nhau. Cụ thể, mô hình sử dụng kiến trúc sâu hơn với nhiều lớp Convolution nhằm tăng khả năng trích xuất đặc trưng từ ảnh CIFAR-10. Bên cạnh đó, Data Augmentation được áp dụng để tăng độ đa dạng của dữ liệu huấn luyện, giúp mô hình tổng quát hóa tốt hơn trên tập kiểm tra.

Các chiến lược cải thiện được sử dụng gồm:

- Sử dụng kiến trúc CNN sâu hơn với 5 lớp Convolution.
- Tăng số lượng filter theo từng tầng: $32 \rightarrow 64 \rightarrow 128$.
- Áp dụng Data Augmentation gồm `RandomHorizontalFlip()` và `RandomRotation(10)`.
- Thêm Dropout với xác suất $p = 0.5$ trong phần classifier.
- Tăng số epoch huấn luyện lên 20.
- Sử dụng Learning Rate = 0.001 với Adam Optimizer.

Các kỹ thuật này giúp mô hình học được nhiều đặc trưng phức tạp hơn, đồng thời hạn chế overfitting và cải thiện khả năng phân loại trên tập test.

4.11.2 Kiến trúc BetterCNN

Mô hình BetterCNN được xây dựng theo hướng tương tự kiến trúc VGG, trong đó nhiều lớp Convolution được xếp chồng liên tiếp trước khi thực hiện Max Pooling. Cách thiết kế này giúp mô hình học được các đặc trưng từ mức thấp đến mức cao, từ các đặc trưng đơn giản như cạnh, màu sắc đến các đặc trưng phức tạp hơn như hình dạng đối tượng.

```
class BetterCNN(nn.Module):
    def __init__(self):
        super(BetterCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(),
            nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.Conv2d(64, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        self.classifier = nn.Sequential(
            nn.Linear(128 * 4 * 4, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, 10)
        )

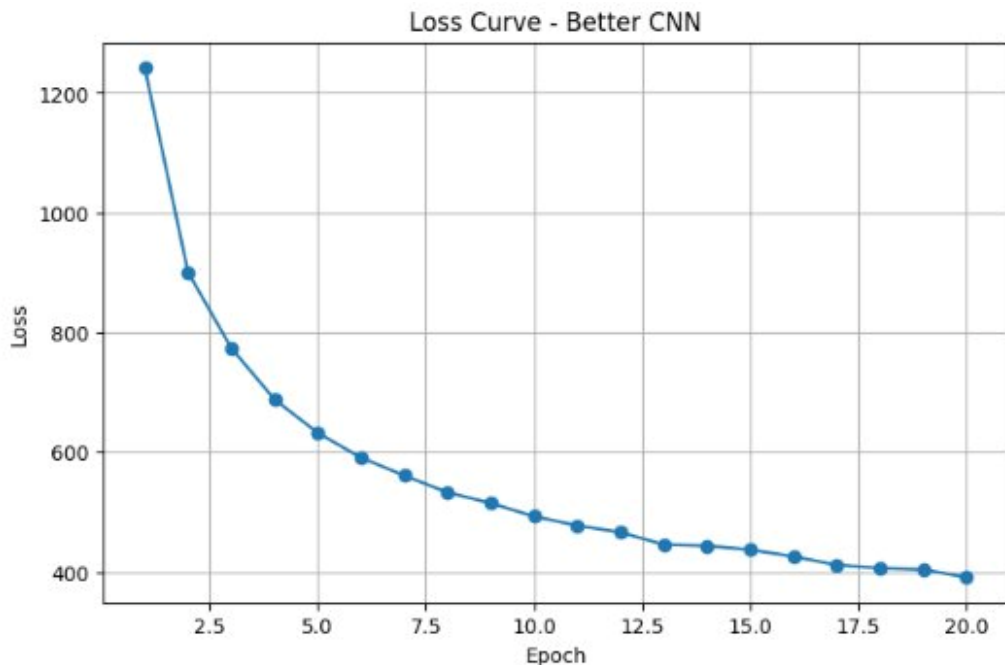
    def forward(self, x):
        x = self.features(x)
        x = x.view(-1, 128 * 4 * 4)
        x = self.classifier(x)
        return x

model_best = BetterCNN().to(device)
```

```
optimizer = optim.Adam(model_best.parameters(), lr=0.001)
# Dùng trainloader_aug (có data augmentation), train 20 epochs
```

Ở phần trích xuất đặc trưng, mô hình gồm các lớp Convolution với số filter lần lượt là 32, 64 và 128. Sau các lớp Convolution, Max Pooling được sử dụng để giảm kích thước feature map. Feature map cuối cùng có kích thước $4 \times 4 \times 128$ và được chuyển thành vector 2048 phần tử thông qua Flatten Layer.

Phần classifier gồm lớp Fully Connected $2048 \rightarrow 512$, hàm kích hoạt ReLU, Dropout(0.5) và lớp đầu ra $512 \rightarrow 10$ tương ứng với 10 lớp của CIFAR-10.



Hình 29: Loss Curve – BetterCNN qua 20 epoch

Nhận xét: Biểu đồ Loss Curve cho thấy giá trị loss giảm liên tục qua 20 epoch. Ở những epoch đầu, loss giảm nhanh, chứng tỏ mô hình học được các đặc trưng cơ bản của dữ liệu. Từ các epoch sau, loss tiếp tục giảm nhưng với tốc độ chậm hơn, cho thấy mô hình dần hội tụ. Điều này chứng minh quá trình huấn luyện diễn ra ổn định và mô hình BetterCNN có khả năng học tốt hơn so với CNN cơ bản.

```

=====
BÀI TẬP 10 - CẢI THIẾN MÔ HÌNH CNN
=====
BetterCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU()
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (5): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (6): ReLU()
    (7): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (8): ReLU()
    (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (10): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU()
    (12): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Linear(in_features=2048, out_features=512, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=512, out_features=10, bias=True)
  )
)

Epoch [1/20] - Loss: 1216.5130
Epoch [2/20] - Loss: 904.8688
Epoch [3/20] - Loss: 758.1860
Epoch [4/20] - Loss: 670.5215
Epoch [5/20] - Loss: 608.9499
Epoch [6/20] - Loss: 568.1325
Epoch [7/20] - Loss: 531.3136
Epoch [8/20] - Loss: 507.7579
Epoch [9/20] - Loss: 486.3113
Epoch [10/20] - Loss: 470.6485
Epoch [11/20] - Loss: 455.9417
Epoch [12/20] - Loss: 441.8285
Epoch [13/20] - Loss: 428.2097
Epoch [14/20] - Loss: 423.5468
Epoch [15/20] - Loss: 409.1290
Epoch [16/20] - Loss: 401.3138
Epoch [17/20] - Loss: 392.8853
Epoch [18/20] - Loss: 389.0646
Epoch [19/20] - Loss: 380.7817
Epoch [20/20] - Loss: 367.8702

```

KẾT QUẢ BETTER CNN

```

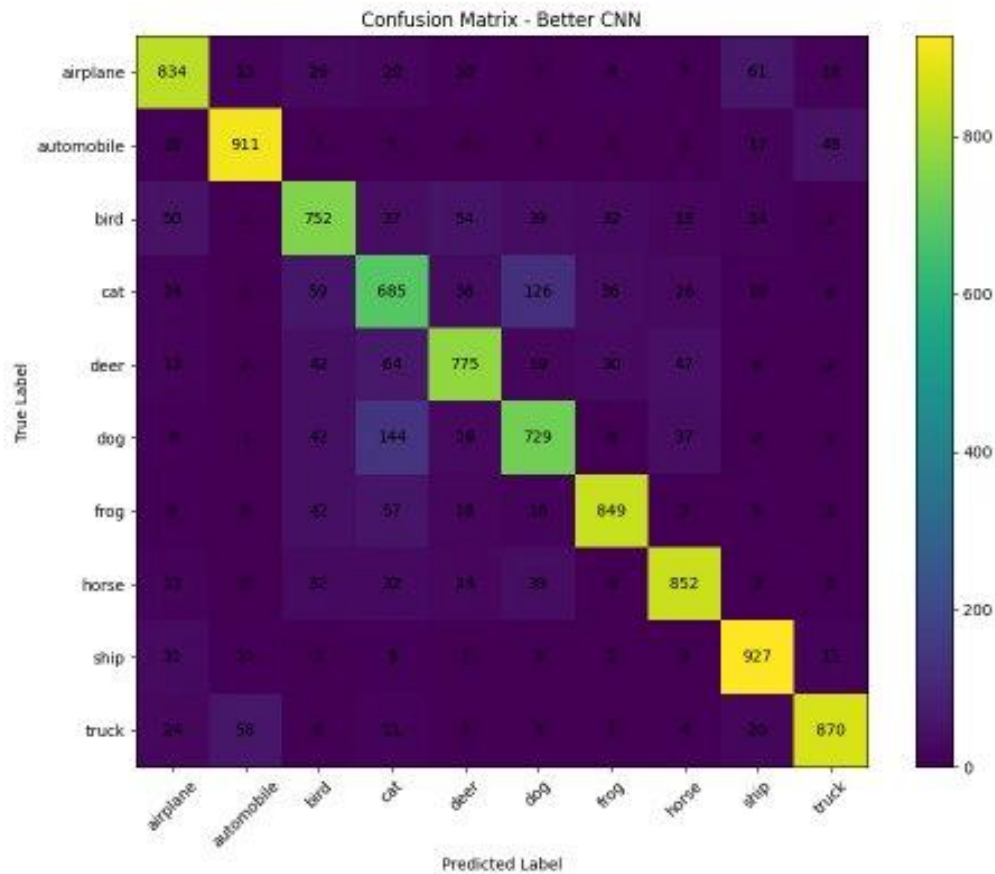
-----
Training time: 520.58 seconds
Test accuracy: 82.00%
Kết luận: Mô hình đạt mục tiêu Accuracy > 75%

```

+ Code

+ Markdown

Hình 30: Loss Curve – BetterCNN



Hình 31: Confusion Matrix – BetterCNN

Nhận xét: Confusion Matrix của BetterCNN cho thấy phần lớn giá trị tập trung trên đường chéo chính, chứng tỏ mô hình phân loại đúng nhiều mẫu dữ liệu. Một số lớp như automobile, frog, horse, ship và truck đạt số lượng dự đoán đúng cao. Tuy nhiên, mô hình vẫn còn nhầm lẫn ở một số lớp có đặc điểm gần giống nhau như cat và dog, deer và horse hoặc automobile và truck. Điều này phù hợp với đặc điểm của bộ dữ liệu CIFAR-10, trong đó ảnh có kích thước nhỏ và nhiều đối tượng có hình dạng tương đồng.

4.11.3 Kết quả đạt được

Sau khi áp dụng các kỹ thuật cải thiện, mô hình BetterCNN đạt Accuracy khoảng 81,84%, vượt mục tiêu yêu cầu là lớn hơn 75%. So với mô hình CNN gốc đạt khoảng 72,49%, BetterCNN cải thiện đáng kể hiệu suất phân loại.

Kết quả này cho thấy việc tăng độ sâu mạng, sử dụng Data Augmentation và bổ sung Dropout giúp mô hình học được đặc trưng tốt hơn và tổng quát hóa hiệu quả hơn trên dữ liệu kiểm tra.

BẢNG TỔNG HỢP KẾT QUẢ THỰC NGHIỆM		
Mô hình / Thí nghiệm	Accuracy (%)	Time (s)
CNN gốc	72.49	135.60
Filter 16->32	72.32	128.78
Filter 32->64	72.87	136.18
Filter 64->128	73.74	155.65
CNN thêm Conv3	76.59	139.45
CNN Dropout	74.71	135.14
Batch size 32	71.40	162.68
Batch size 64	71.41	143.08
Batch size 128	73.56	134.55
Learning rate 0.01	50.16	134.12
Learning rate 0.001	72.64	134.03
Learning rate 0.0001	64.27	133.85
Data Augmentation	75.67	219.21
Better CNN	81.84	521.03

Hình 32: Bảng tổng hợp kết quả thực nghiệm

Bảng 8: Bảng tổng hợp kết quả tất cả thực nghiệm

Mô hình / Thí nghiệm	Test Accuracy (%)	Thời gian (s)	Ghi chú
CNN gốc (32→64 filters)	72,49	135,60	Baseline
Filter 16→32	72,32	128,78	Ít filter
Filter 64→128	73,74	155,65	Nhiều filter
CNN + Conv3 (3 layers)	76,59	139,45	Sâu hơn
CNN + Dropout(0.5)	74,71	135,14	Regularization
Batch size = 32	71,40	162,68	Nhỏ nhất
Batch size = 64	71,41	143,08	Mặc định
Batch size = 128	73,56	134,55	Lớn nhất

Mô hình / Thí nghiệm	Test Accuracy (%)	Thời gian (s)	Ghi chú
lr = 0.01	50,16	134,12	Quá lớn
lr = 0.001	72,64	134,03	Tối ưu
lr = 0.0001	64,27	133,85	Quá nhỏ
Data Augmentation	75,67	219,21	Flip + Rotate
BetterCNN (20 epochs)	81,84	521,03	Mô hình tốt nhất

Nhận xét:

Bảng tổng hợp cho thấy BetterCNN là mô hình có độ chính xác cao nhất trong các thí nghiệm. Trong khi CNN gốc chỉ đạt khoảng 72,49%, các cải tiến như tăng số filter, thêm lớp Convolution, sử dụng Dropout và Data Augmentation đều giúp cải thiện hiệu quả mô hình ở các mức độ khác nhau. Đặc biệt, Data Augmentation và kiến trúc BetterCNN mang lại mức cải thiện rõ rệt nhất.

Tuy nhiên, BetterCNN cũng có thời gian huấn luyện lớn hơn do mô hình sâu hơn và sử dụng nhiều lớp Convolution hơn. Vì vậy, có thể thấy việc cải thiện Accuracy thường đi kèm với việc tăng chi phí tính toán. Trong phạm vi bài thực hành, BetterCNN đạt được sự cải thiện tốt nhất về độ chính xác và đáp ứng yêu cầu Accuracy > 75%.

5. THẢO LUẬN VÀ ĐÁNH GIÁ

5.1 Ảnh hưởng của số Filter đến hiệu suất

Kết quả thực nghiệm cho thấy số lượng filter ảnh hưởng trực tiếp đến khả năng trích xuất đặc trưng của mô hình CNN. Khi tăng số lượng filter từ cấu hình 16→32 lên 64→128, Accuracy tăng từ 72,32% lên 73,74%. Điều này cho thấy nhiều filter hơn giúp mô hình học được nhiều đặc trưng hình ảnh hơn, đặc biệt đối với bộ dữ liệu CIFAR-10 có màu sắc và hình dạng đa dạng.

Tuy nhiên, số lượng filter lớn cũng làm tăng số lượng tham số và thời gian huấn luyện. Mức cải thiện Accuracy giữa các cấu hình không quá lớn, cho thấy việc tăng filter cần được cân nhắc giữa hiệu quả phân loại và chi phí tính toán.

5.2 Ảnh hưởng của số lớp Convolution

Việc bổ sung thêm một lớp Convolution giúp Accuracy tăng từ 72,49% lên 76,59%, chứng tỏ mô hình sâu hơn có khả năng học được các đặc trưng phức tạp hơn từ dữ liệu ảnh. Các lớp Convolution đầu học các đặc trưng cơ bản như cạnh và màu sắc, trong khi các lớp sâu hơn học được các đặc trưng có mức độ trừu tượng cao hơn.

Kết quả này cho thấy việc tăng độ sâu mạng là một hướng cải tiến hiệu quả đối với bài toán phân loại ảnh CIFAR-10. Tuy nhiên, nếu sử dụng quá nhiều lớp Pooling, kích thước Feature Map sẽ giảm nhanh và có thể làm mất thông tin quan trọng của ảnh.

5.3 Hiệu quả của Dropout

Khi bổ sung Dropout(0.5), Accuracy tăng từ 72,49% lên 74,71%. Điều này cho thấy Dropout giúp mô hình giảm hiện tượng overfitting và cải thiện khả năng tổng quát hóa trên tập kiểm tra.

Dropout đặc biệt hiệu quả đối với các lớp Fully Connected có số lượng tham số lớn. Bằng cách vô hiệu hóa ngẫu nhiên một phần neuron trong quá trình huấn luyện, mô hình học được các đặc trưng ổn định hơn và ít phụ thuộc vào các neuron riêng lẻ.

5.4 Ảnh hưởng của Batch Size

Kết quả thực nghiệm cho thấy Batch Size ảnh hưởng đến cả Accuracy và thời gian huấn luyện. Trong ba giá trị khảo sát, Batch Size = 128 đạt Accuracy cao nhất (73,56%) đồng thời có thời gian huấn luyện ngắn nhất (134,55 giây).

Ngược lại, Batch Size = 32 có thời gian huấn luyện dài nhất do số lần cập nhật trọng số nhiều hơn. Kết quả này cho thấy đối với mô hình và cấu hình phần cứng được sử dụng trong bài thực hành, Batch Size lớn mang lại hiệu quả tốt hơn.

5.5 Ảnh hưởng của Learning Rate

Learning Rate ảnh hưởng trực tiếp đến tốc độ hội tụ của mô hình. Với Learning Rate = 0.01, giá trị Loss giảm nhanh nhất trong 10 epoch nhưng có nguy cơ dao động nếu tiếp tục huấn luyện lâu hơn. Learning Rate = 0.0001 làm quá trình học diễn ra rất chậm và chưa hội tụ tốt sau 10 epoch.

Trong thực nghiệm, Learning Rate = 0.001 cho quá trình huấn luyện ổn định và được lựa chọn để xây dựng mô hình cải tiến BetterCNN. Đây cũng là giá trị thường được sử dụng cùng Adam Optimizer trong các bài toán phân loại ảnh.

5.6 Hiệu quả của Data Augmentation

Data Augmentation là một trong những kỹ thuật cải thiện hiệu quả nhất trong bài thực hành. Accuracy tăng từ 72,49% lên 75,67% sau khi áp dụng RandomHorizontalFlip và RandomRotation(10).

Các phép biến đổi này giúp tăng độ đa dạng của dữ liệu huấn luyện mà không cần thu thập thêm ảnh mới. Nhờ đó, mô hình học được các đặc trưng tổng quát hơn và giảm sự phụ thuộc vào các mẫu dữ liệu gốc, từ đó cải thiện khả năng phân loại trên tập kiểm tra.

5.7 Phân tích Confusion Matrix

Confusion Matrix cho thấy mô hình phân loại tốt các lớp như automobile, ship, frog và horse với số lượng dự đoán đúng cao. Ngược lại, các lớp cat và dog vẫn có tỷ lệ nhầm lẫn đáng kể do có nhiều đặc điểm hình dạng tương đồng.

Ngoài ra, một số trường hợp nhầm lẫn cũng xuất hiện giữa automobile và truck hoặc deer và horse. Điều này phản ánh độ khó của bộ dữ liệu CIFAR-10 khi các ảnh chỉ có kích thước 32×32 pixel và chứa nhiều chi tiết nền phức tạp.

Nhìn chung, Confusion Matrix cho thấy mô hình đã học được các đặc trưng quan trọng của dữ liệu, tuy nhiên vẫn còn khó khăn trong việc phân biệt các lớp có đặc điểm trực quan gần giống nhau.

5.8 Tổng hợp cấu hình tốt nhất

Bảng 9: Tổng hợp cấu hình tối ưu

Siêu tham số	Giá trị tối ưu	Các giá trị đã thử	Lý do lựa chọn
Số Filter	64→128	16→32, 32→64, 64→128	Accuracy cao nhất trong nhóm filter
Số Conv Layers	5 (BetterCNN)	2, 3, 5	Học được đặc trưng sâu và phức tạp hơn
Dropout	0.5	Không, 0.5	Giảm overfitting và cải thiện tổng quát hóa
Batch Size	128	32, 64, 128	Accuracy cao nhất và thời gian huấn luyện thấp
Learning Rate	0.001	0.01, 0.001, 0.0001	Hội tụ ổn định, phù hợp với Adam
Data Augmentation	Flip + Rotate(10)	Không, Flip+Rotate	Cải thiện khả năng tổng quát hóa

Siêu tham số	Giá trị tối ưu	Các giá trị đã thử	Lý do lựa chọn
Epochs	20	10, 20	Mô hình hội tụ tốt hơn, Accuracy cao hơn
Test Accuracy	81,84%	—	Kết quả tốt nhất đạt được

Bảng 9 tổng hợp các siêu tham số và kỹ thuật cho kết quả tốt nhất trong quá trình thực nghiệm. Kết quả cho thấy việc kết hợp nhiều giải pháp cải tiến như tăng số lượng filter, sử dụng kiến trúc CNN sâu hơn, áp dụng Data Augmentation và bổ sung Dropout đã giúp nâng cao đáng kể hiệu suất mô hình.

Mô hình BetterCNN với 5 lớp Convolution, Dropout(0.5), Data Augmentation, Learning Rate = 0.001 và huấn luyện trong 20 epoch đạt Accuracy 81,84%, cao hơn khoảng 9,35% so với mô hình CNN gốc (72,49%). Điều này chứng tỏ việc tối ưu đồng thời kiến trúc mạng và các siêu tham số mang lại hiệu quả rõ rệt hơn so với việc chỉ thay đổi một yếu tố riêng lẻ.

Trong các kỹ thuật khảo sát, Data Augmentation và việc tăng số lớp Convolution là hai yếu tố đóng góp đáng kể nhất vào sự cải thiện độ chính xác của mô hình. Kết quả cuối cùng đã vượt mục tiêu đề ra là đạt Accuracy lớn hơn 75% trên bộ dữ liệu CIFAR-10.

5.9 Hạn chế và hướng cải thiện

Mặc dù mô hình BetterCNN đã đạt Accuracy 81,84%, vượt mục tiêu đề ra (>75%), bài toán phân loại ảnh CIFAR-10 vẫn còn nhiều thách thức. Một số lớp như cat và dog hoặc automobile và truck vẫn có tỷ lệ nhầm lẫn tương đối cao do đặc điểm hình ảnh tương đồng và kích thước ảnh nhỏ (32×32 pixel).

Trong tương lai, có thể tiếp tục cải thiện mô hình theo các hướng sau:

- Bổ sung Batch Normalization sau các lớp Convolution để tăng tốc độ hội tụ và ổn định quá trình huấn luyện.

- Sử dụng Learning Rate Scheduler (StepLR hoặc CosineAnnealingLR) để tự động điều chỉnh Learning Rate trong quá trình huấn luyện.
- Áp dụng thêm các kỹ thuật Data Augmentation như RandomCrop, ColorJitter hoặc Cutout nhằm tăng độ đa dạng của dữ liệu.
- Thử nghiệm các kiến trúc CNN hiện đại hơn như VGG, ResNet hoặc DenseNet.
- Sử dụng Transfer Learning từ các mô hình pretrained trên ImageNet để cải thiện hiệu suất phân loại.
- Huấn luyện với số epoch lớn hơn hoặc thực hiện Hyperparameter Tuning để tìm kiếm cấu hình tối ưu hơn.

Các hướng cải tiến trên được kỳ vọng sẽ tiếp tục nâng cao Accuracy và khả năng tổng quát hóa của mô hình trên bộ dữ liệu CIFAR-10.

6. KẾT LUẬN

6.1 Tóm tắt kết quả đạt được

Trong bài thực hành này, mô hình Convolutional Neural Network (CNN) đã được xây dựng và triển khai bằng thư viện PyTorch nhằm giải quyết bài toán phân loại ảnh trên bộ dữ liệu CIFAR-10. Thông qua quá trình huấn luyện, đánh giá và tối ưu hóa, mô hình đã đạt được hiệu quả phân loại tốt, đồng thời giúp làm rõ vai trò của các thành phần quan trọng trong kiến trúc CNN.

Quá trình thực nghiệm bao gồm đầy đủ các nội dung từ khảo sát dữ liệu, phân tích kiến trúc CNN cơ bản, đánh giá ảnh hưởng của số lượng filter, số lớp convolution, Dropout, Batch Size, Learning Rate, Data Augmentation, Confusion Matrix đến việc xây dựng mô hình cải tiến. Trên cơ sở các kết quả thu được, mô hình BetterCNN đã được đề xuất với kiến trúc sâu hơn, kết hợp Data Augmentation và Dropout nhằm nâng cao khả năng học đặc trưng và giảm hiện tượng overfitting.

Kết quả cuối cùng cho thấy mô hình BetterCNN đạt độ chính xác kiểm tra (Test Accuracy) khoảng **81,84%**, cao hơn đáng kể so với mô hình CNN gốc (**72,49%**) và vượt mục tiêu đề ra là đạt Accuracy lớn hơn 75% trên bộ dữ liệu CIFAR-10.

6.2 Bài học rút ra

Thông qua quá trình thực hiện bài thực hành, một số kết luận quan trọng có thể được rút ra như sau:

- CNN cho hiệu quả vượt trội hơn các mạng neural truyền thống như MLP trong các bài toán xử lý ảnh nhờ khả năng khai thác cấu trúc không gian của dữ liệu thông qua các lớp Convolution và cơ chế chia sẻ trọng số.
- Số lượng filter và số lớp Convolution ảnh hưởng trực tiếp đến khả năng trích xuất đặc trưng của mô hình. Việc tăng độ sâu mạng giúp mô hình học được các đặc trưng có mức độ trừu tượng cao hơn.

- Learning Rate đóng vai trò quan trọng đối với tốc độ hội tụ và độ ổn định của quá trình huấn luyện. Trong thực nghiệm, Learning Rate = 0.001 kết hợp với Adam Optimizer cho kết quả ổn định và hiệu quả nhất.
- Dropout và Data Augmentation là những kỹ thuật hiệu quả trong việc giảm overfitting và cải thiện khả năng tổng quát hóa của mô hình trên dữ liệu chưa từng xuất hiện.
- Confusion Matrix cung cấp cái nhìn chi tiết về hiệu suất phân loại trên từng lớp dữ liệu, giúp xác định các nhóm đối tượng dễ bị nhầm lẫn và đánh giá mô hình toàn diện hơn so với Accuracy đơn thuần.
- CIFAR-10 là bộ dữ liệu có độ phức tạp cao hơn MNIST do sử dụng ảnh màu RGB và chứa nhiều lớp đối tượng có đặc điểm hình ảnh tương đồng, đòi hỏi mô hình phải học được các đặc trưng phong phú hơn.

6.3 Hướng phát triển tiếp theo

Mặc dù mô hình BetterCNN đã đạt được kết quả khả quan, vẫn còn nhiều hướng nghiên cứu và cải tiến có thể tiếp tục thực hiện trong tương lai:

- Bổ sung Batch Normalization vào kiến trúc mạng nhằm tăng tốc độ hội tụ và nâng cao độ ổn định trong quá trình huấn luyện.
- Nghiên cứu và triển khai các kiến trúc CNN hiện đại hơn như VGG, ResNet, DenseNet hoặc EfficientNet để cải thiện hiệu suất phân loại.
- Áp dụng Transfer Learning từ các mô hình pretrained trên ImageNet nhằm tận dụng tri thức đã học và rút ngắn thời gian huấn luyện.
- Khảo sát thêm các kỹ thuật Data Augmentation nâng cao như Random Crop, Color Jitter, Cutout hoặc Mixup để tăng khả năng tổng quát hóa của mô hình.
- Mở rộng nghiên cứu sang các bộ dữ liệu có độ phức tạp cao hơn như CIFAR-100 hoặc các tập dữ liệu thực tế có số lượng lớp lớn hơn.
- Tiếp tục phát triển theo hướng các bài toán thị giác máy tính nâng cao như Object Detection, Instance Segmentation hoặc Semantic Segmentation dựa trên nền tảng CNN đã được xây dựng trong bài thực hành.

Nhìn chung, bài thực hành đã giúp củng cố kiến thức về Deep Learning và Computer Vision, đồng thời cung cấp kinh nghiệm thực tế trong việc thiết kế, huấn luyện, đánh giá và tối ưu hóa các mô hình CNN cho bài toán phân loại ảnh.